

CSE4067

Introduction to Blockchain Programming

Goshgar Can Ismayilov

goshgar.ismayilov@boun.edu.tr

<https://www.linkedin.com/in/gismayilov/>

12.11.2024

Atomicity of Transaction

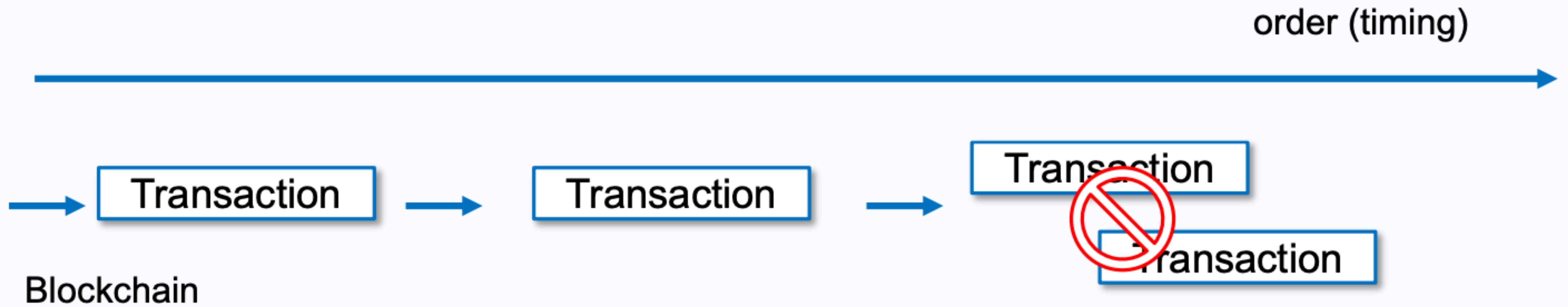


A transaction is **an atomic operation**. Can't divide or interrupt.

Transaction or **Transaction**

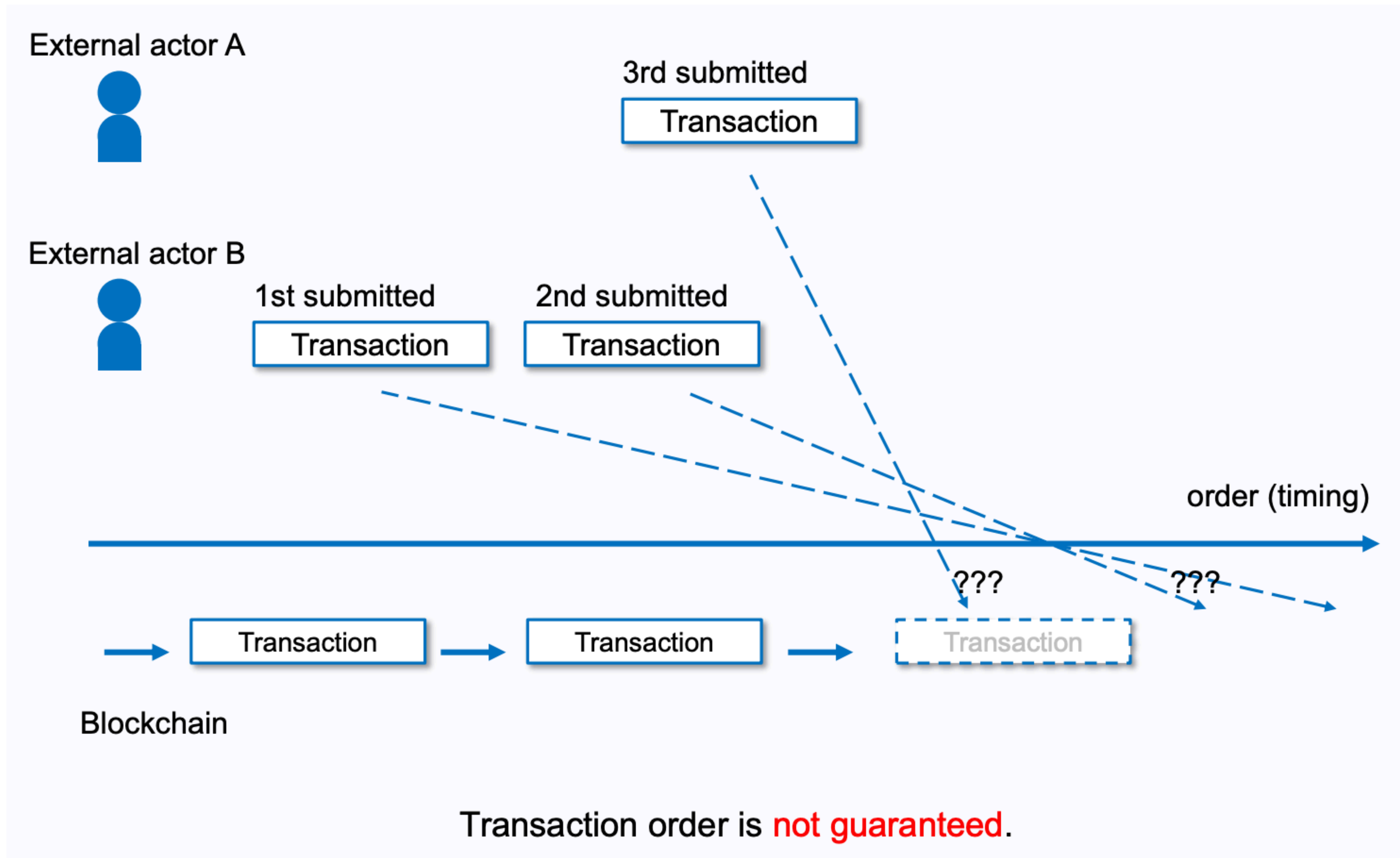
That is, **All** (complete done) or **Nothing** (zero effect).

Order of Transaction

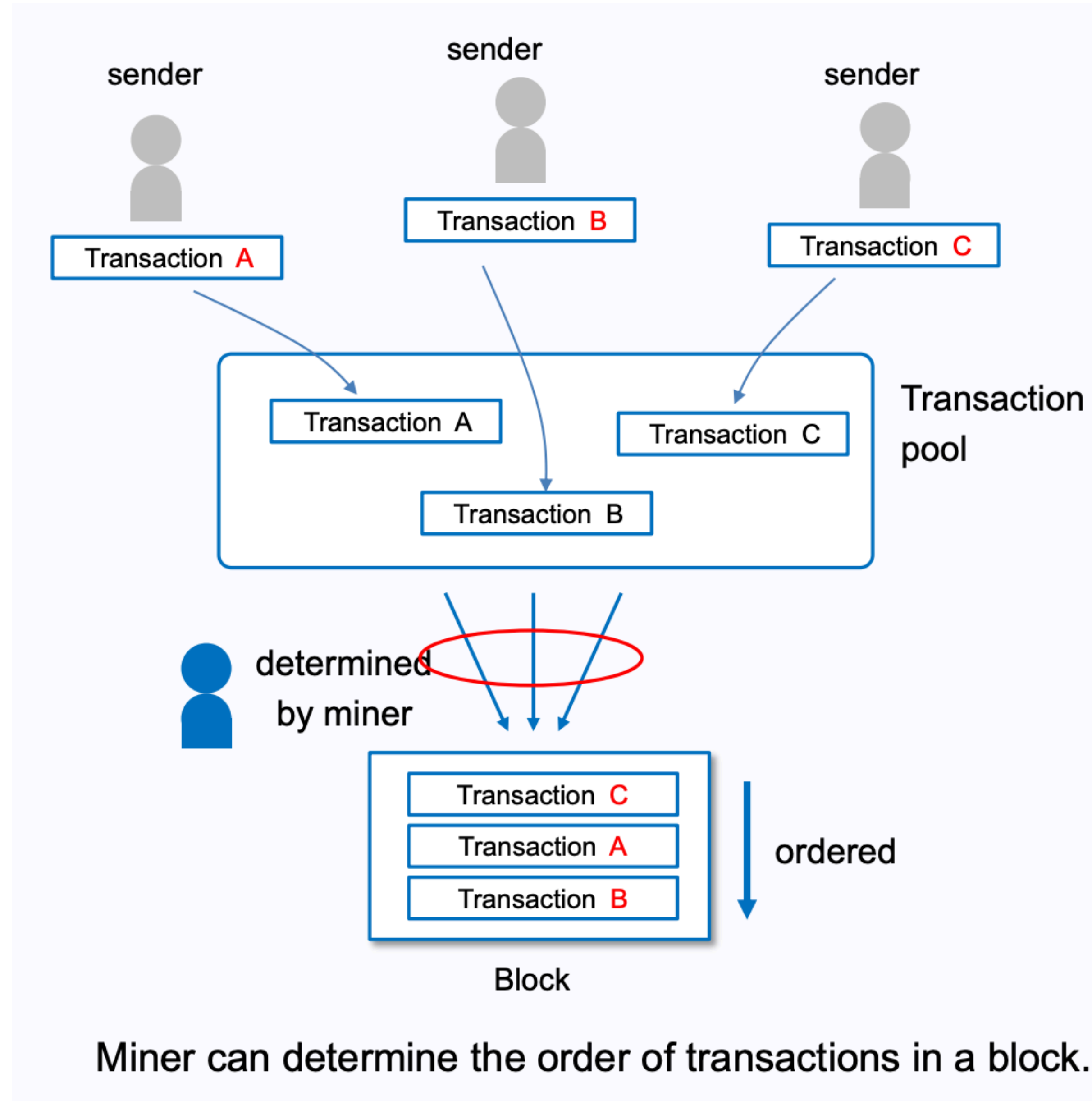


Transactions **cannot be overlapped**.
Transactions must be executed sequentially.

Order of Transaction



Order of Transaction



Events

Events notify applications about changes made to contracts and applications which can be used to execute dependent logic. Solidity events allow us to easily query “stuff” that happened in blocks and transactions. If you run a blockchain node, you can “listen” for Solidity events and react accordingly. Knowing how to use events in Solidity makes smart contract development easier.

```
interface IERC20 {  
    event Transfer(address indexed from, address indexed to, uint256 value);  
}
```

Events

```
function _transfer(  
    address from,  
    address to,  
    uint256 amount  
) internal virtual {  
    // perform various checks, such as the `from` address has `amount` of tokens  
  
    // do the transfer of tokens  
    unchecked {  
        _balances[from] = fromBalance - amount;  
        _balances[to] += amount;  
    }  
  
    // the Transfer event is emitted here  
    emit Transfer(from, to, amount);  
  
    // perform various cleanup  
}
```

Events

```
const provider = new ethers.providers.Web3Provider(window.ethereum);  
const contract = new Contract(erc20TokenAddress, ERC20_ABI, provider);  
  
contract.on('Transfer', async (from, to, amount, data) => {  
  console.log('Transfer event emitted. {from, to, amount, data}');  
});
```

Ethereum Token Standards

Tokens are smart contracts that implement a standardized interface. They are frequently used by many decentralized applications running on the Ethereum ecosystem.

Depending on the actual use case, different token standards exist. The most common use cases and standards are:

- Decentralized Finance, ICOs, crowd funding (mostly ERC 20)
- Gaming (mostly ERC 20 and ERC 721)

Token classes:

1. Fungible tokens: All tokens are indistinguishable and have exactly the same value.



2. Non-fungible tokens (NFT): Each token is uniquely identifiable by an ID.



3. ERC-1155 Multi-token standard: Allows for each token ID to represent a new configurable token type, which may have its own metadata, supply and other attributes.



ERC-20 Token Standard

The **ERC20** standard was first proposed by Fabian Vogelsteller and Vitalik Buterin in November 2015. The specification **defines an interface** that a contract must implement to be ERC20 compliant. It does **not specify** the **actual implementation** of the functions.

It is the most commonly used token standard in the Ethereum network with more than 450,000 contracts deployed on the main network. Each **ERC20 contract** defines a class of **fungible tokens**.

Interface

```
interface IERC20 {  
    function totalSupply() public view returns (uint);  
    function balanceOf(address tokenOwner) public view returns (uint balance);  
    function allowance(address tokenOwner, address spender) public view returns (uint remaining);  
    function transfer(address to, uint tokens) public returns (bool success);  
    function approve(address spender, uint tokens) public returns (bool success);  
    function transferFrom(address from, address to, uint tokens) public returns (bool success);  
  
    event Transfer(address indexed from, address indexed to, uint tokens);  
    event Approval(address indexed tokenOwner, address indexed spender, uint tokens);  
}
```

ERC-20

```
contract MyERCToken {  
    string public constant name = "My Custom ERC20 Token";  
}
```

```
contract MyERCToken {  
    string public constant symbol = "MET";  
}
```

```
contract MyERCToken {  
    uint8 public constant decimals = 18;  
}
```

ERC-20

```
contract MyERCToken {  
    // In this case, the total supply  
    // of MyERCToken is fixed, but  
    // it can very much be changed  
    uint256 _totalSupply = 1000000;  
  
    function totalSupply() constant returns (uint256 theTotalSupply) {  
        // Because our function signature  
        // states that the returning variable  
        // is "theTotalSupply", we'll just set that variable  
        // to the value of the instance variable "_totalSupply"  
        // and return it  
        theTotalSupply = _totalSupply;  
        return theTotalSupply;  
    }  
}
```

ERC-20

```
contract MyERCToken {  
    // Create a table so that we can map addresses  
    // to the balances associated with them  
    mapping(address => uint256) balances;  
    // Owner of this contract  
    address public owner;  
  
    function balanceOf(address _owner) constant returns (uint256 balance) {  
        // Return the balance for the specific address  
        return balances[_owner];  
    }  
}
```

ERC-20

```
contract MyERCToken {  
    // Create a table so that we can map  
    // the addresses of contract owners to  
    // those who are allowed to utilize the owner's contract  
    mapping(address => mapping (address => uint256)) allowed;  
  
    function approve(address _spender, uint256 _amount) returns (bool success) {  
        allowed[msg.sender][_spender] = _amount;  
        // Fire the event "Approval" to execute any logic  
        // that was listening to it  
        Approval(msg.sender, _spender, _amount);  
        return true;  
    }  
}
```

ERC-20

```
contract MyERCToken {
    mapping(address => uint256) balances;

    // Note: This function returns a boolean value
    //       indicating whether the transfer was successful
    function transfer(address _to, uint256 _amount) returns (bool success) {
        // If the sender has sufficient funds to send
        // and the amount is not zero, then send to
        // the given address
        if (balances[msg.sender] >= _amount
            && _amount > 0
            && balances[_to] + _amount > balances[_to]) {
            balances[msg.sender] -= _amount;
            balances[_to] += _amount;
            // Fire a transfer event for any
            // logic that's listening
            Transfer(msg.sender, _to, _amount);
            return true;
        } else {
            return false;
        }
    }
}
```

ERC-20

```
contract MyERCToken {
    mapping(address => uint256) balances;

    function transferFrom(address _from, address _to, uint256 _amount) returns (bool success) {
        if (balances[_from] >= _amount
            && allowed[_from][msg.sender] >= _amount
            && _amount > 0
            && balances[_to] + _amount > balances[_to]) {
            balances[_from] -= _amount;
            balances[_to] += _amount;
            Transfer(_from, _to, _amount);
            return true;
        } else {
            return false;
        }
    }
}
```

ERC-721 Token Standard

Interface

```
pragma solidity ^0.4.20;

/// @title ERC-721 Non-Fungible Token Standard
/// @dev See https://eips.ethereum.org/EIPS/eip-721

interface ERC721 {
    function balanceOf(address _owner) external view returns (uint256);
    function ownerOf(uint256 _tokenId) external view returns (address);
    function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes data) external payable;
    function safeTransferFrom(address _from, address _to, uint256 _tokenId) external payable;
    function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
    function approve(address _approved, uint256 _tokenId) external payable;
    function setApprovalForAll(address _operator, bool _approved) external;
    function getApproved(uint256 _tokenId) external view returns (address);
    function isApprovedForAll(address _owner, address _operator) external view returns (bool);

    event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId);
    event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId);
    event ApprovalForAll(address indexed _owner, address indexed _operator, bool _approved);
}
```

ERC-721

name

This function is used to tell outside contracts and applications the name of this token. An example implementation of the function can be as follows.

```
contract MyNFT {  
    function name() constant returns (string name){  
        return "My Non-Fungible Token";  
    }  
}
```

ERC-721

symbol

This function also helps in providing compatibility with the ERC20 token standard. It provides outside programs with the token's shorthand name, or symbol.

```
contract MyNFT {  
    function symbol() constant returns (string symbol){  
        return "MNFT";  
    }  
}
```

ERC-721

totalSupply

This function returns the total number of coins available on the blockchain.
The supply does not have to be constant.

```
contract MyNFT {  
    // This can be an arbitrary number  
    uint256 private totalSupply = 10000000000;  
  
    function totalSupply() constant returns (uint256 supply){  
        return totalSupply;  
    }  
}
```

ERC-721

balanceOf

This function is used to find the number of tokens that a given address owns.

```
contract MyNFT {  
    mapping(address => uint) private balances;  
  
    function balanceOf(address _owner) constant returns (uint balance)  
    {  
        return balances[_owner];  
    }  
}
```

ERC-721

ownerOf

This function returns the address of the owner of a token. Because each ERC721 token is non-fungible and, therefore, unique, it's referenced on the blockchain via a unique ID. We can determine the owner of a token using its ID.

```
contract MyNFT {  
    mapping(uint256 => address) private tokenOwners;  
    mapping(uint256 => bool) private tokenExists;  
  
    function ownerOf(uint256 _tokenId)  
    constant returns (address owner) {  
        require(tokenExists[_tokenId]);  
  
        return tokenOwners[_tokenId];  
    }  
}
```

ERC-721

approve

This function approves, or grants, another entity permission to transfer a token on the owner's behalf. For example, if Alice owns 1 MyNFT she can call the `approve` function for her friend Bob. After a successful call, Bob could take ownership of or perform operations on the token at a later time on Alice's behalf. More information on transferring ownership can be seen in the `takeOwnership` and `transfer` functions.

```
contract MyNFT {
    mapping(address => mapping (address => uint256)) allowed;

    function approve(address _to, uint256 _tokenId){
        require(msg.sender == ownerOf(_tokenId));
        require(msg.sender != _to);

        allowed[msg.sender][_to] = _tokenId;
        Approval(msg.sender, _to, _tokenId);
    }
}
```

ERC-721

takeOwnership

This function acts like a *withdraw* function, since an outside party can call it in order to take tokens out of another user's account. Therefore, `takeOwnership` can be used to when a user has been *approved* to own a certain amount of tokens and wishes to withdraw said tokens from another user's balance.

```
contract MyNFT {  
    function takeOwnership(uint256 _tokenId){  
        require(tokenExists[_tokenId]);  
  
        address oldOwner = ownerOf(_tokenId);  
        address newOwner = msg.sender;  
  
        require(newOwner != oldOwner);  
  
        require(allowed[oldOwner][newOwner] == _tokenId);  
        balances[oldOwner] -= 1;  
        tokenOwners[_tokenId] = newOwner;  
  
        balances[newOwner] += 1;  
        Transfer(oldOwner, newOwner, _tokenId);  
    }  
}
```

ERC-721

transfer

The next method of transferring tokens is using this function. `transfer` lets the owner of a token send it to another user, similar to a standalone cryptocurrency. However, a transfer can only be initiated if the receiving account has previously been approved to own the token by the sending account.

ERC-721

```
contract MyNFT {
    mapping(address => mapping(uint256 => uint256)) private
    ownerTokens;
    function removeFromTokenList(address owner, uint256 _tokenId)
    private {
        for(uint256 i = 0;ownerTokens[owner][i] != _tokenId;i++){
            ownerTokens[owner][i] = 0;
        }
    }

    function transfer(address _to, uint256 _tokenId){
        address currentOwner = msg.sender;
        address newOwner = _to;

        require(tokenExists[_tokenId]);

        require(currentOwner == ownerOf(_tokenId));
        require(currentOwner != newOwner);
        require(newOwner != address(0));
        removeFromTokenList(_tokenId);

        balances[oldOwner] -= 1;
        tokenOwners[_tokenId] = newOwner;

        balances[newOwner] += 1;
        Transfer(oldOwner, newOwner, _tokenId);
    }
}
```

ERC-721

tokenOfOwnerByIndex (Optional — Recommended)

Each non-fungible token owner can own more than one token at one time. Because each token is referenced by its unique ID, however, it can get difficult to keep track of the individual tokens that a user may own. To do this, the contract keeps a record of the IDs of each token that each user owns. Because of this, each individual token owned by a user can be retrieved by its index in the list (array) of tokens owned by the user.

`tokenOfOwnerByIndex` lets us retrieve a token in this method.

```
contract MyNFT {
    mapping(address => mapping(uint256 => uint256)) private
    ownerTokens;

    function tokenOfOwnerByIndex(address _owner, uint256 _index)
    constant returns (uint tokenId){

        return ownerTokens[_owner][_index];

    }
}
```

ERC-721

tokenMetadata (Optional — Recommended)

This function lets us discover a token's metadata, or the link to the its data.

```
contract MyNFT {  
    mapping(uint256 => string) tokenLinks;  
  
    function tokenMetadata(uint256 _tokenId) constant returns (string  
infoUrl) {  
        return tokenLinks[_tokenId];  
    }  
}
```

ERC-721

Transfer

This event is fired whenever a token changes hands. It's broadcasted when a token's ownership moves from one user to another. It details which account sent the token, which account received the token, and which token (by ID) was transferred.

```
contract MyNFT {  
    event Transfer(address indexed _from, address indexed _to, uint256  
        _tokenId);  
}
```

ERC-721

Approval

This event is fired whenever a user approves another user to take ownership of a token (i.e. whenever `approve` is executed). It details which account currently owns the token, which account is allowed to own the token in the future, and which token (by ID) is approved to have its ownership transferred.

```
contract MyNFT {  
    event Approval(address indexed _owner, address indexed _approved,  
        uint256 _tokenId);  
}
```

Ethereum Virtual Machine (EVM)

code

The code basically represents a smart contract as bytecode. For the EVM, a smart contract is a sequence of instructions (called *opcodes*) similar to assembly code.

Example:

```
PUSH1 0x60  
PUSH1 0x40  
MSTORE  
PUSH1 0x04  
CALLDATASIZE  
LT  
PUSH2 0x00b6  
JUMPI  
PUSH4 0xffffffff
```

memory

An infinitely expandable byte array that is non-persistent and used as temporal storage during execution.

EVM

stack

The stack is also used as a fast, non-persistent buffer to which 32 byte values can be pushed and popped during execution.

pc

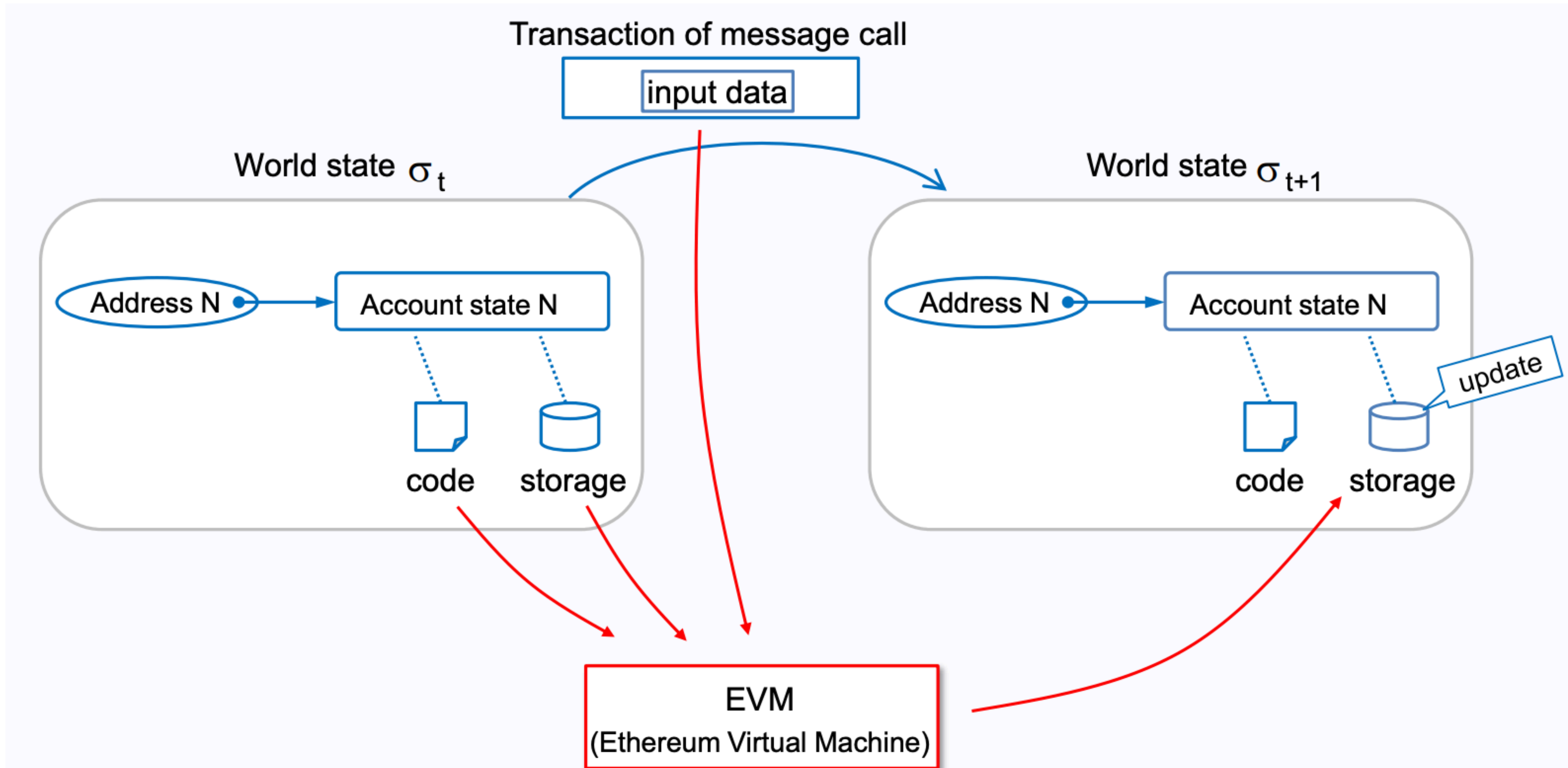
PC stands for “program counter”. The program counter is always initialized with 0 and points to the position of the current opcode instruction.

Simple Opcode Execution Example:

0	<i>PUSH1 0x60</i>
1	<i>PUSH1 0x40</i>
2	<i>MSTORE</i>
3	<i>PUSH1 0x04</i>
4	<i>CALLDATASIZE</i>
5	<i>LT</i>
6	<i>PUSH2 0x00b6</i>
7	<i>JUMPI</i>
8	<i>PUSH4 0xffffffff</i>



EVM



EVM code is executed on Ethereum Virtual Machine (EVM).

EVM

Code

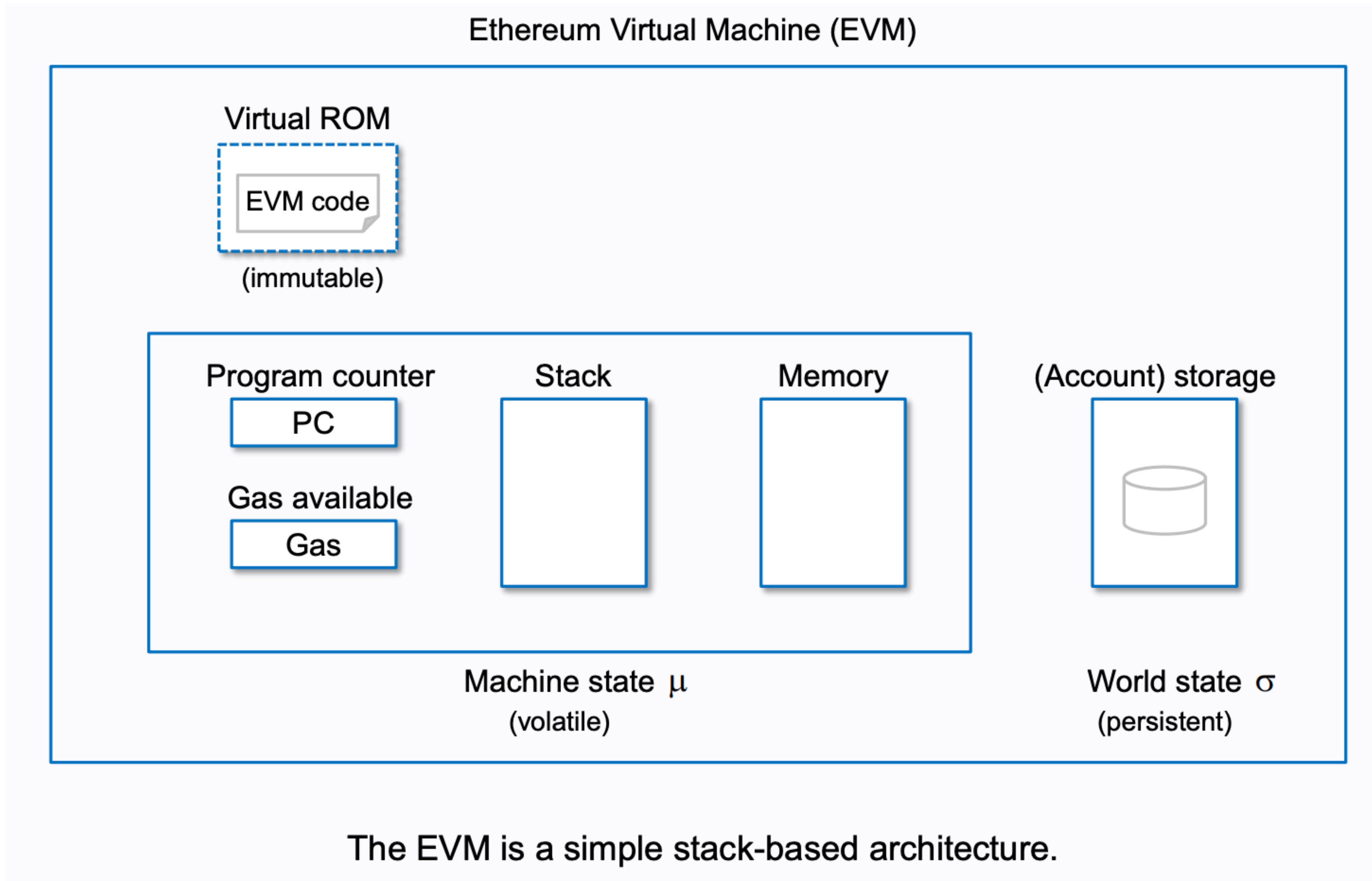
EVM code

Virtual machine

EVM (Ethereum Virtual Machine)

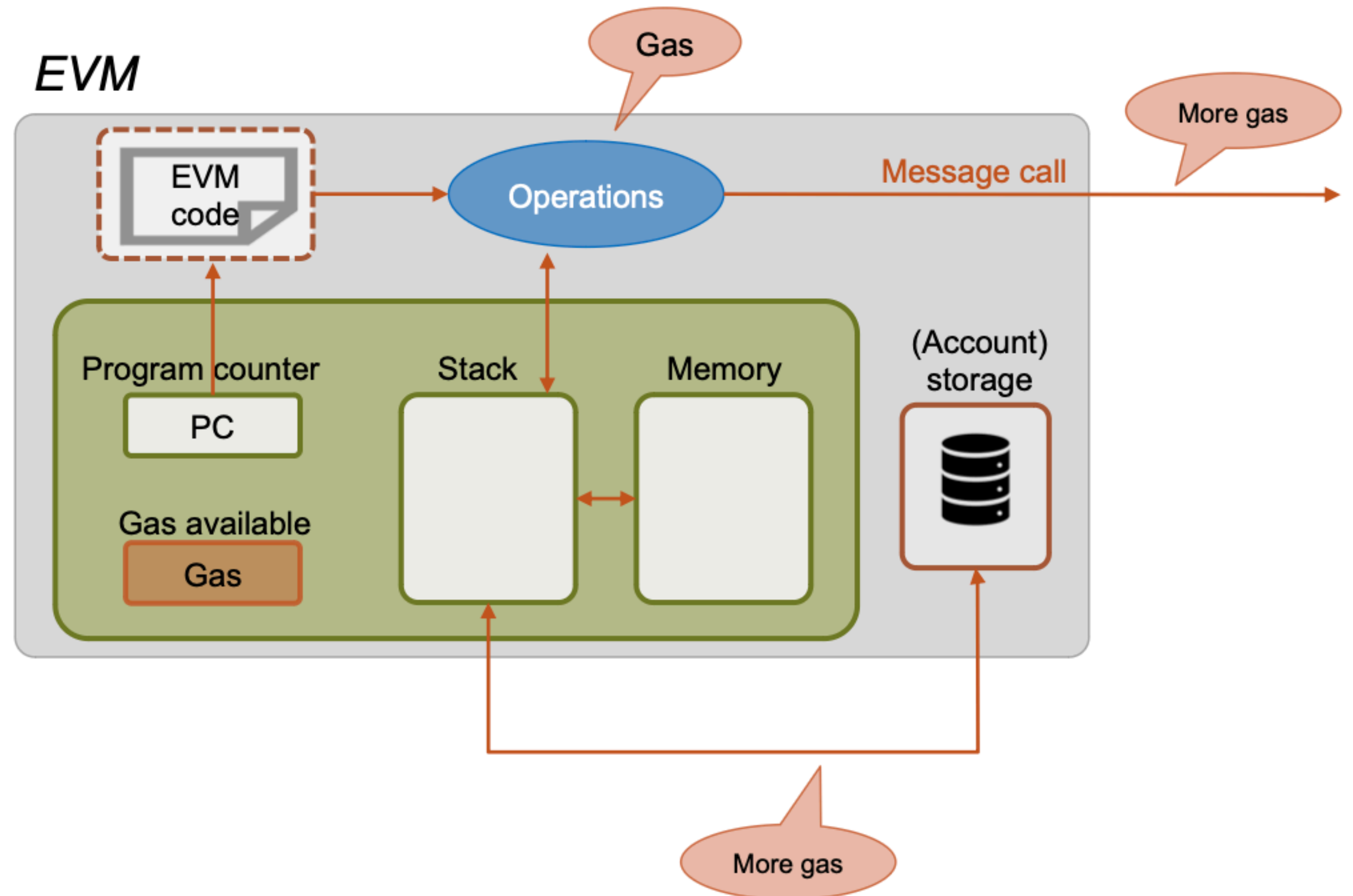
The Ethereum Virtual Machine is the runtime environment for smart contracts in Ethereum.

EVM



EVM

- Executing EVM *opcodes* use computational resources and therefore **has an associated fee** called **gas**.
- Each opcode costs a certain amount of gas which may depend on the arguments of the operation, e.g., the number of bytes to be allocated.
- Opcodes such as **SSTORE**, which places data into the storage, are more expensive than simple operations like **ADD** or **PUSH**.¹
- The opcode for **SELFDESTRUCT (address)** uses **negative gas** as it frees up space on the blockchain.²



EVM

Stack



stack memory

256 bits x 1024 elements

Memory



volatile memory

byte addressing
linear memory

(Account) storage

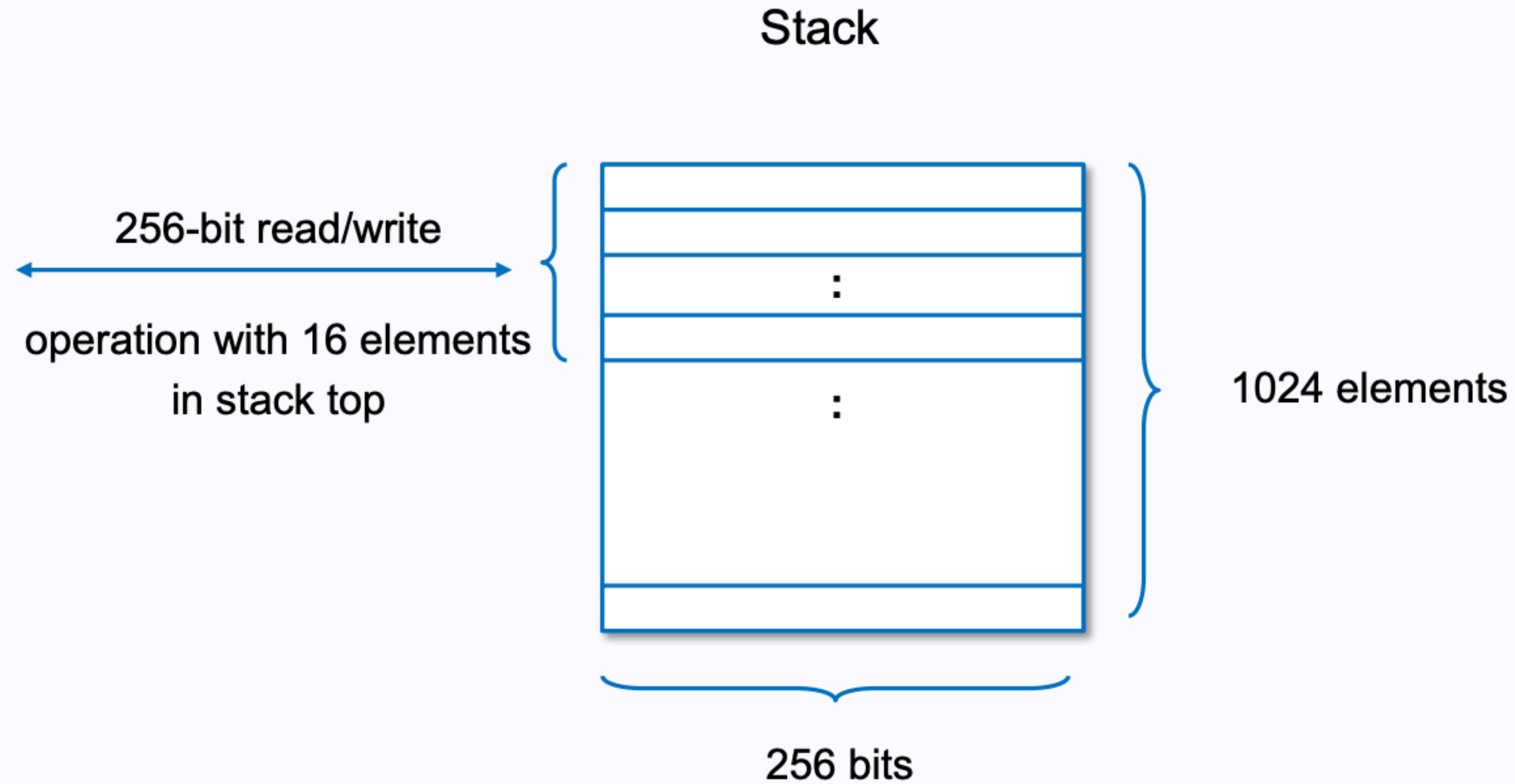


persistent memory

256 bits to 256 bits
key-value store

There are several resources as space.

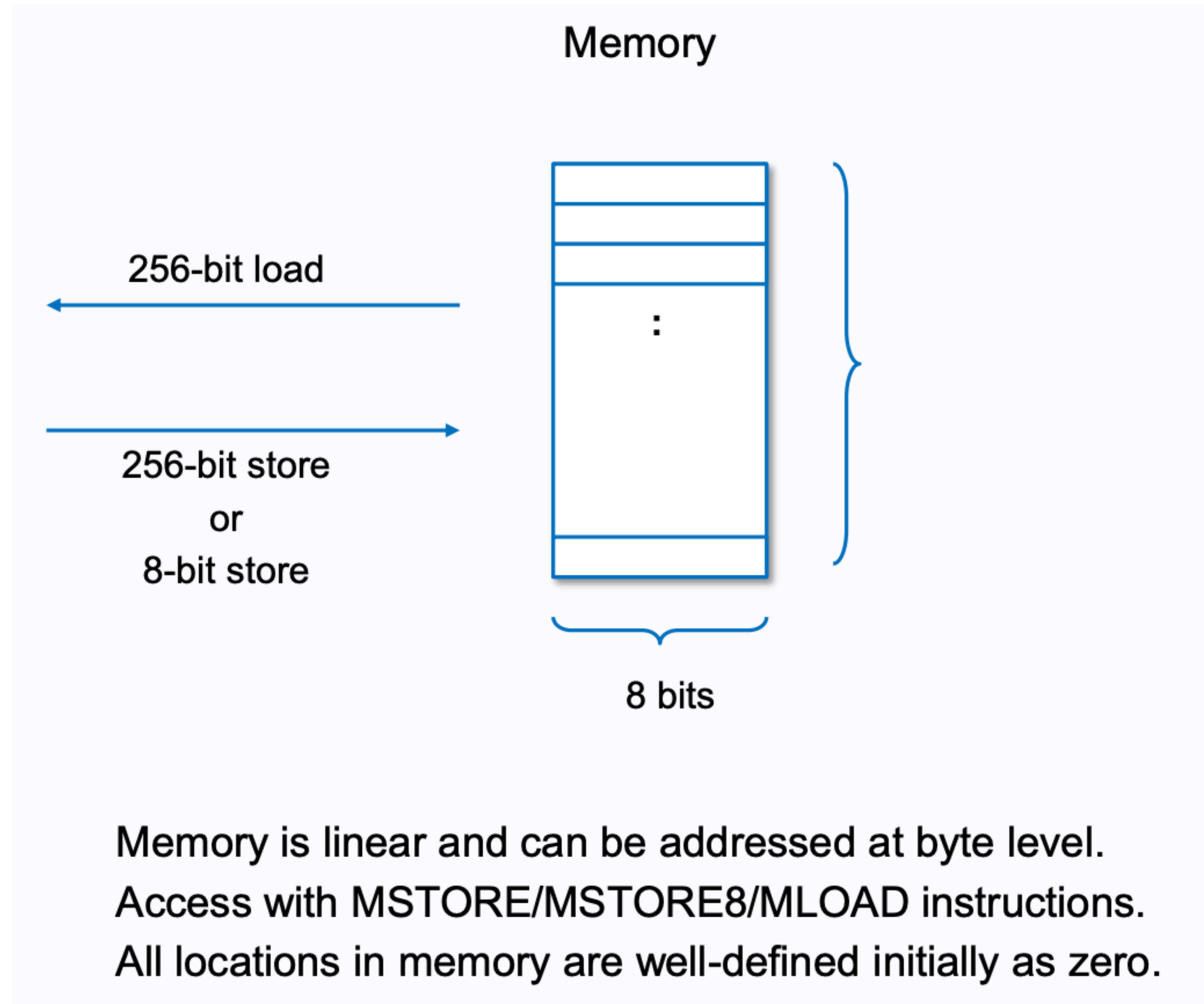
EVM



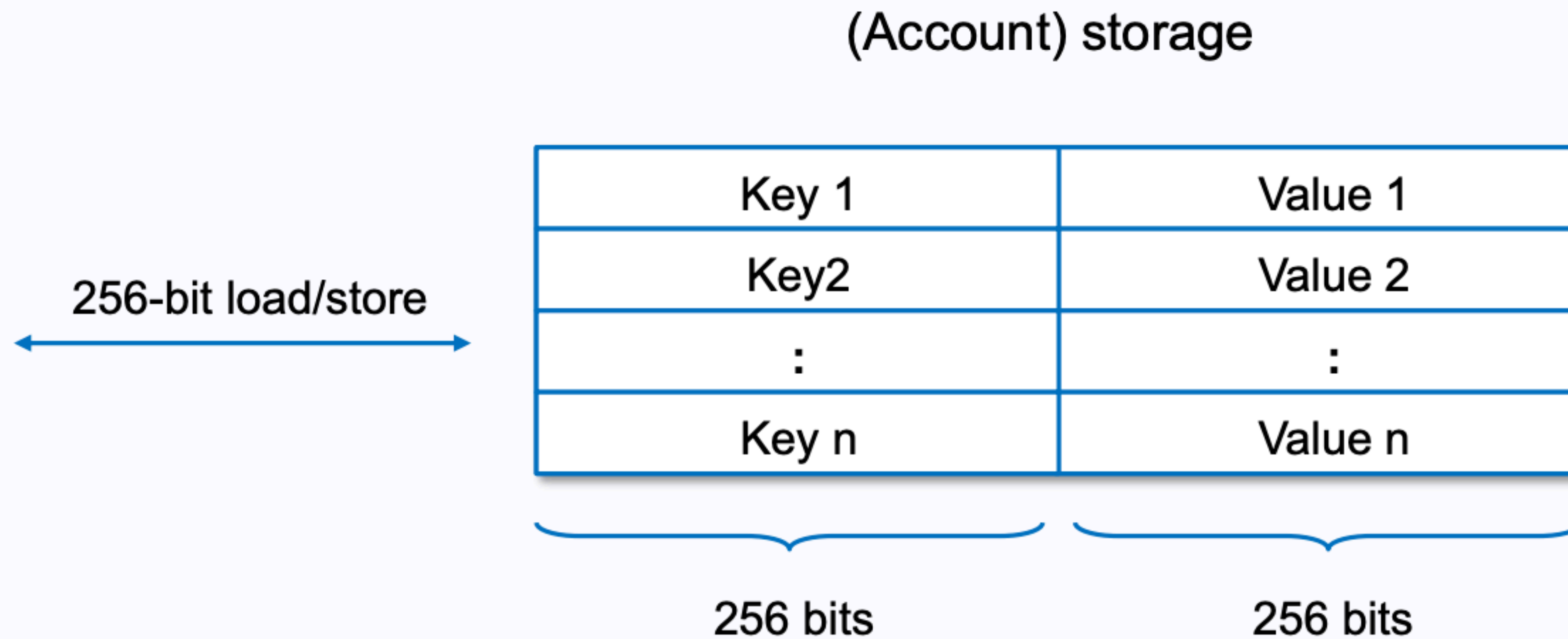
All operation are performed on the stack.

Access with many instructions such as PUSH/POP/COPY/SWAP, ...

EVM



EVM



Storage is a key-value store that maps 256-bit words to 256-bit words.
Access with `SSTORE/SLOAD` instructions.
All locations in storage are well-defined initially as zero.

EVM

Assembly view

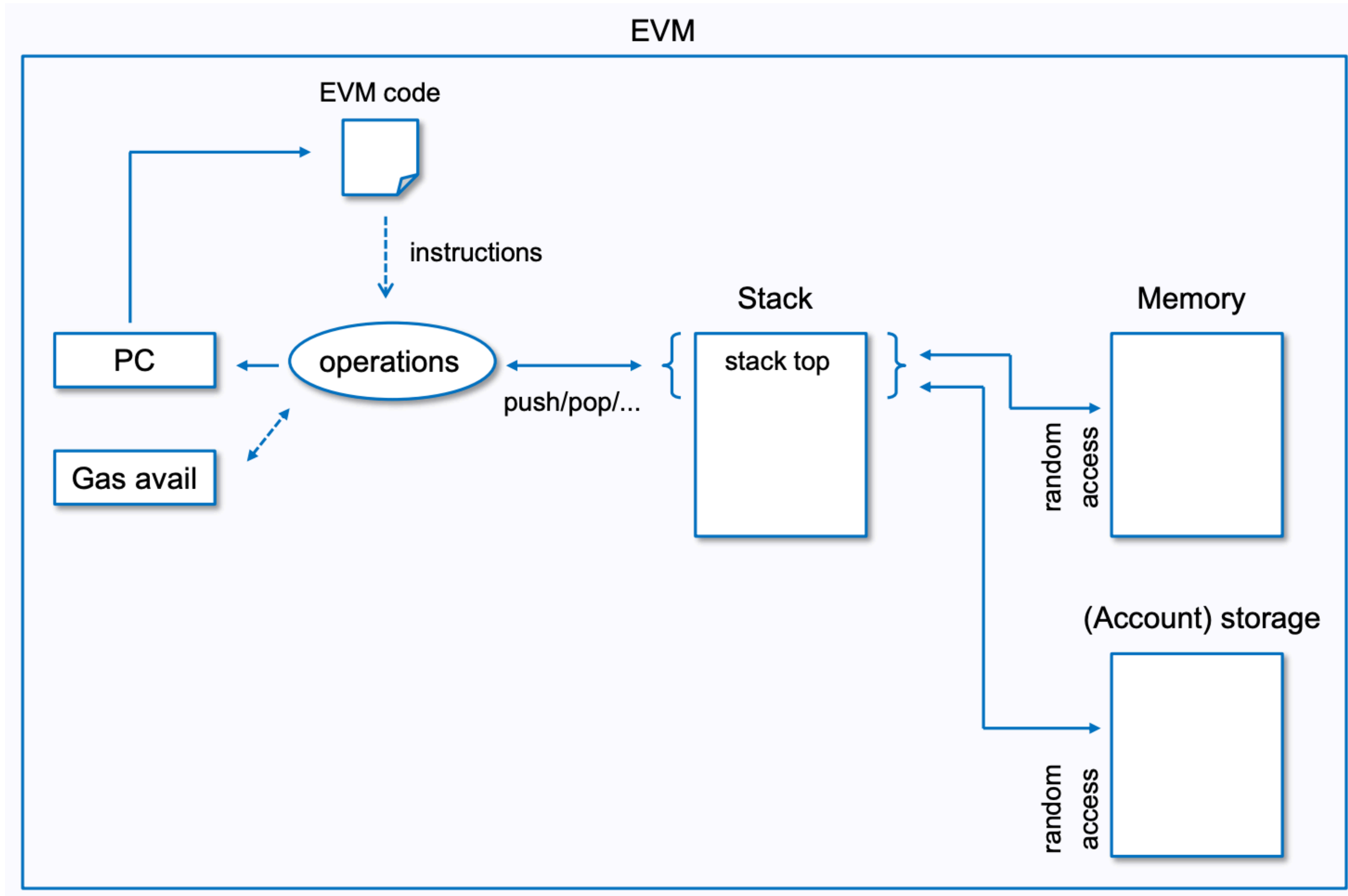
```
PUSH1 e0  
PUSH1 02  
EXP  
PUSH1 00  
CALLDATALOAD  
:
```

Bytecode view

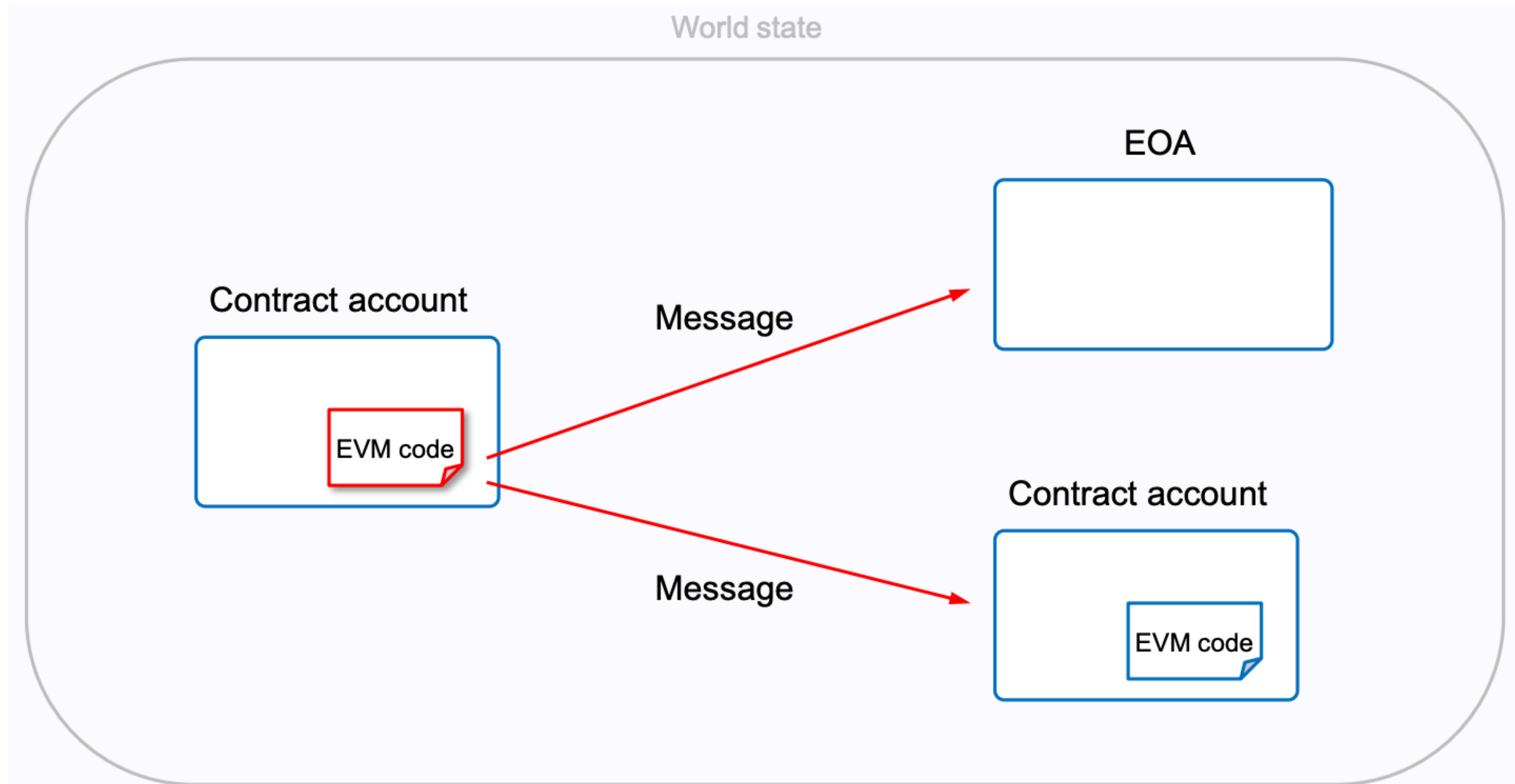
```
0x60e060020a600035...
```

EVM Code is the bytecode that the EVM can natively execute.

EVM



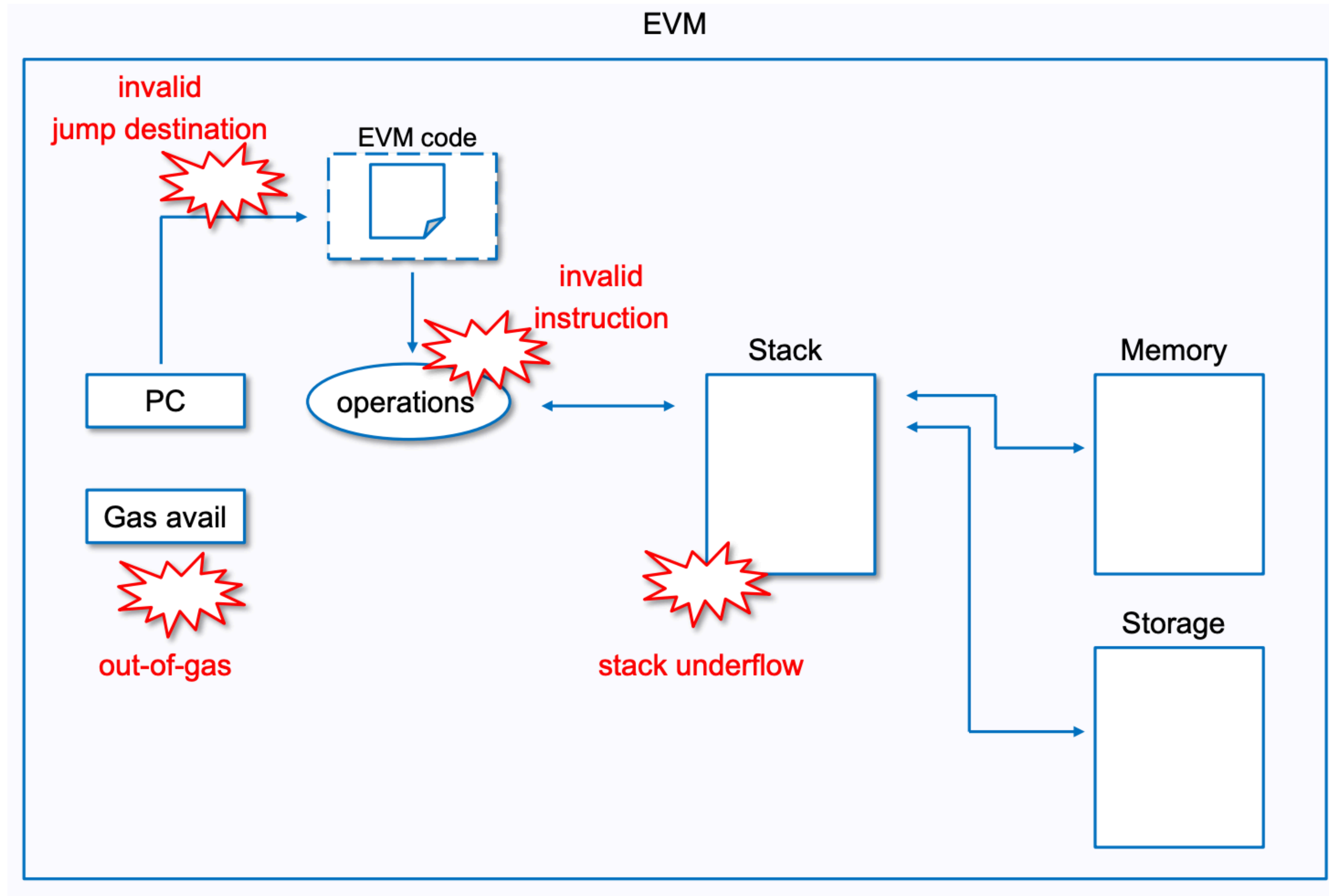
EVM



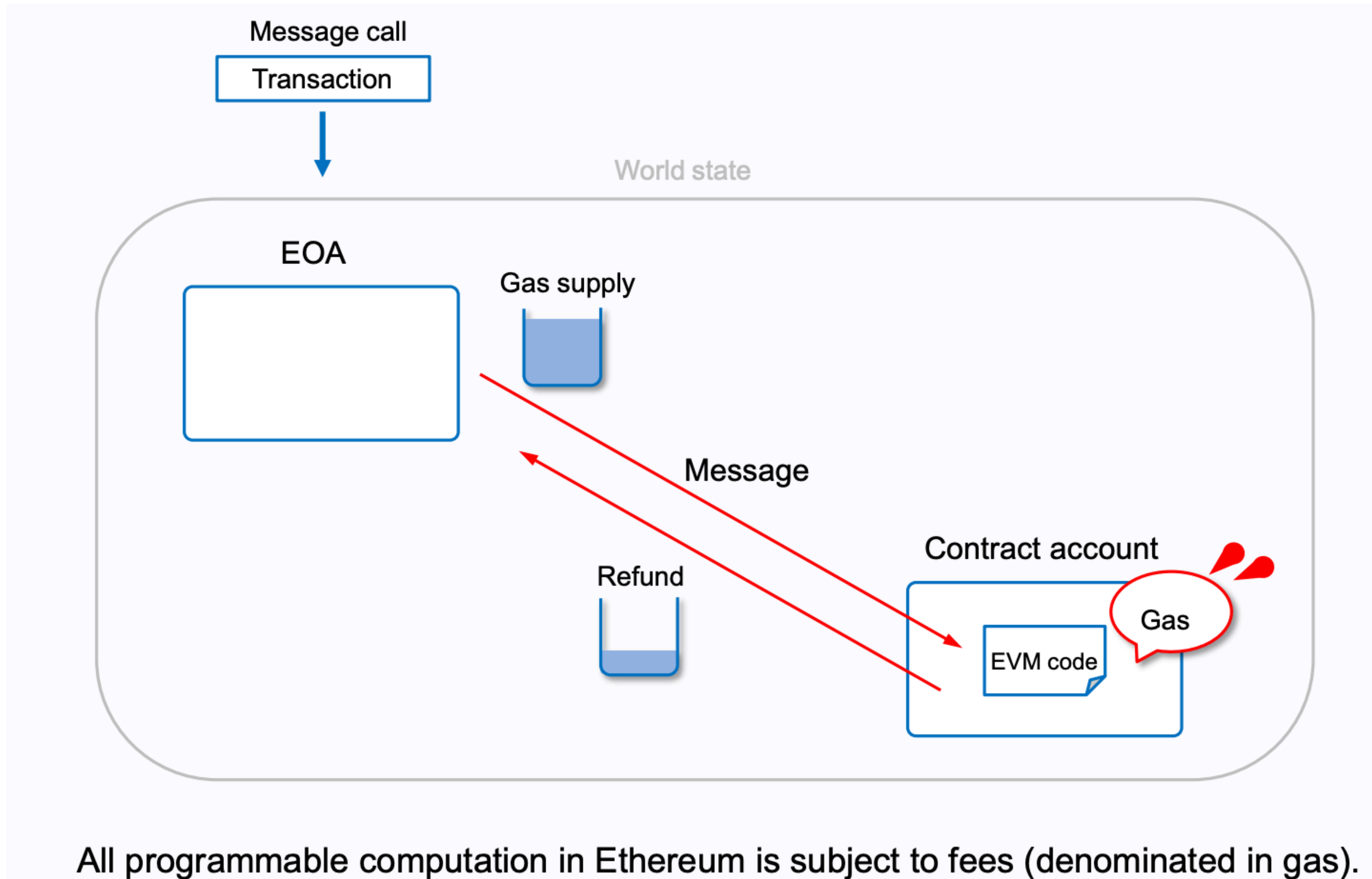
EVM can send a message to other account.

The depth of message call is limited to less than 1024 levels.

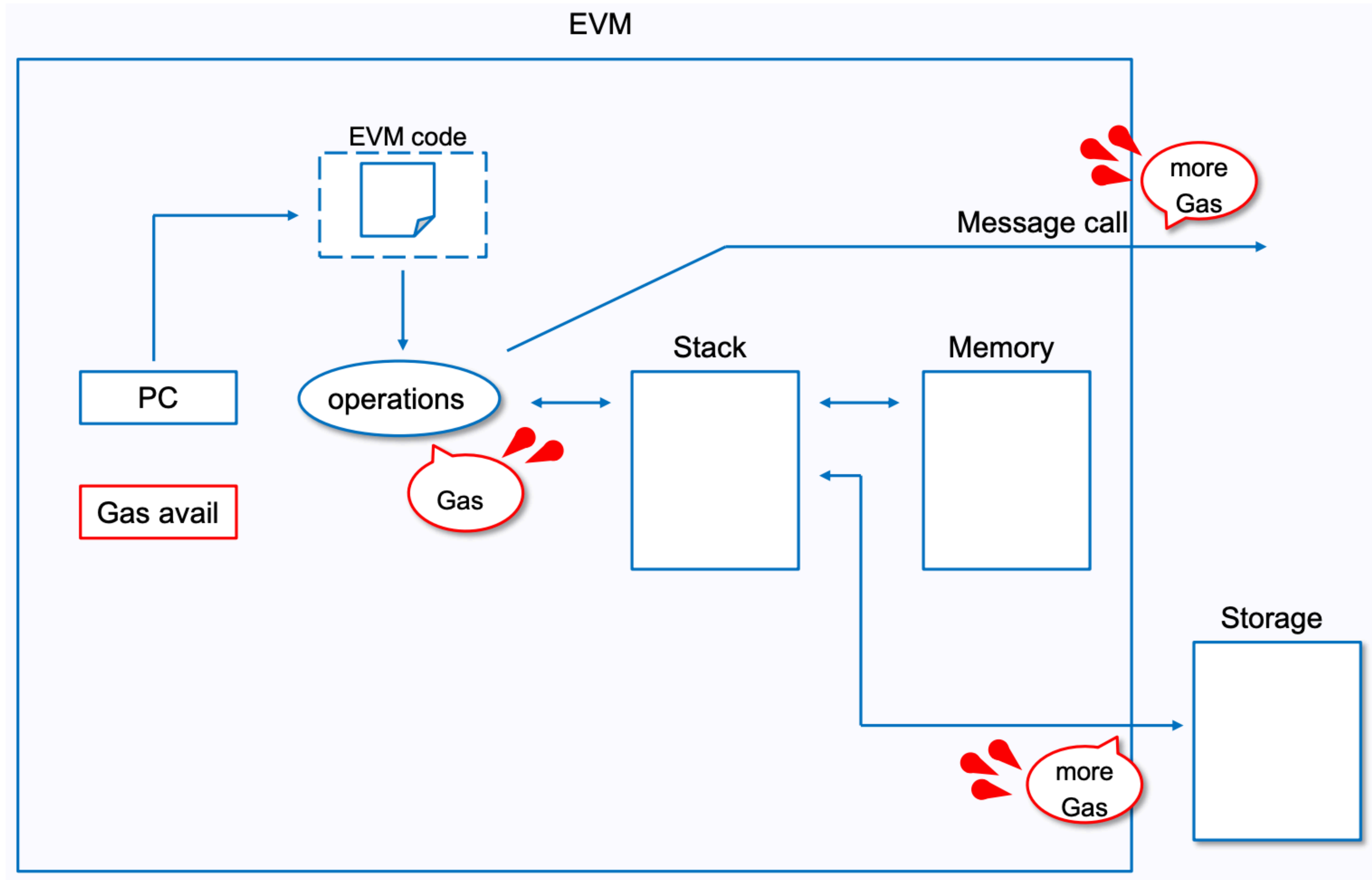
EVM - Exceptions



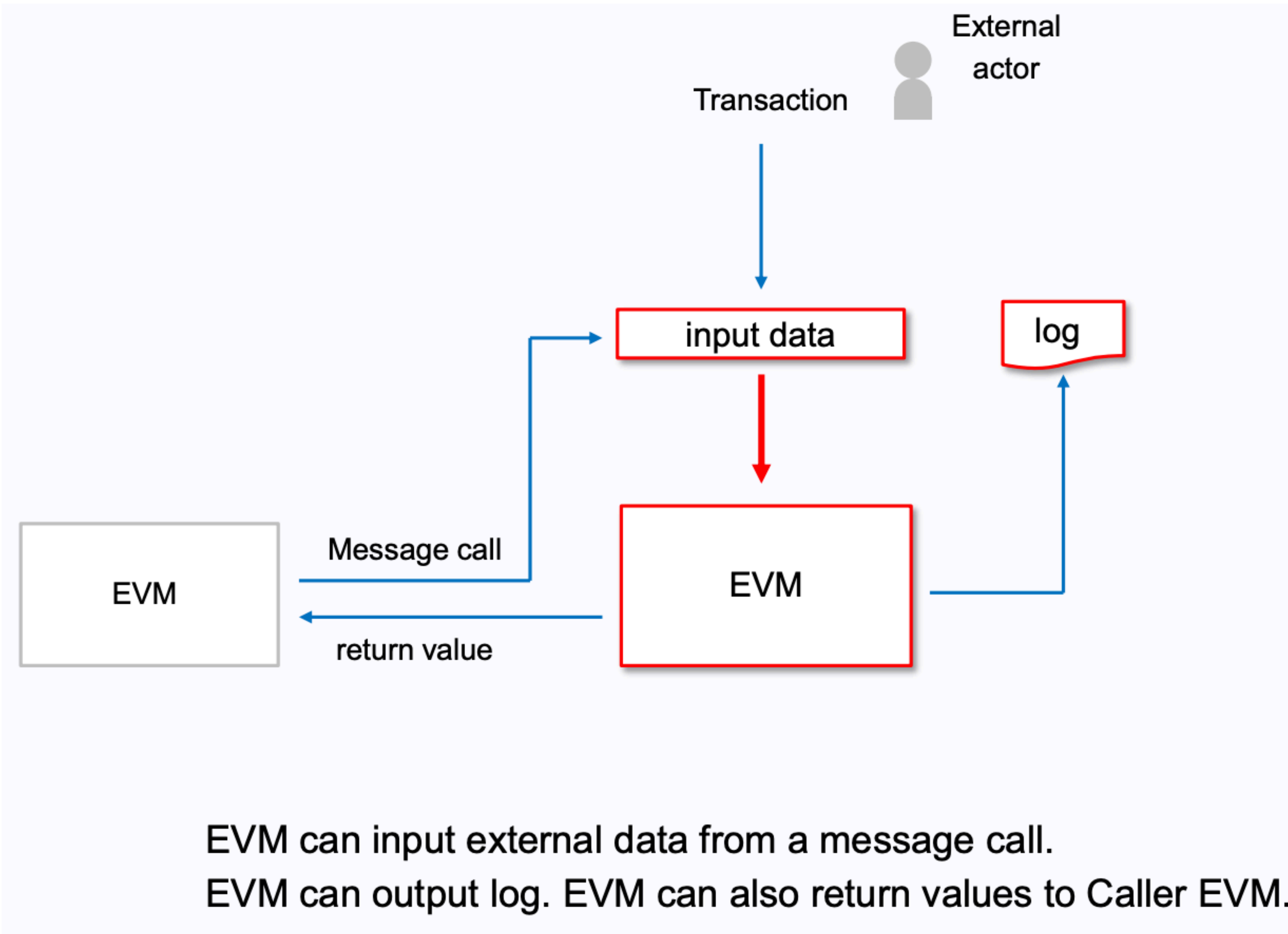
EVM



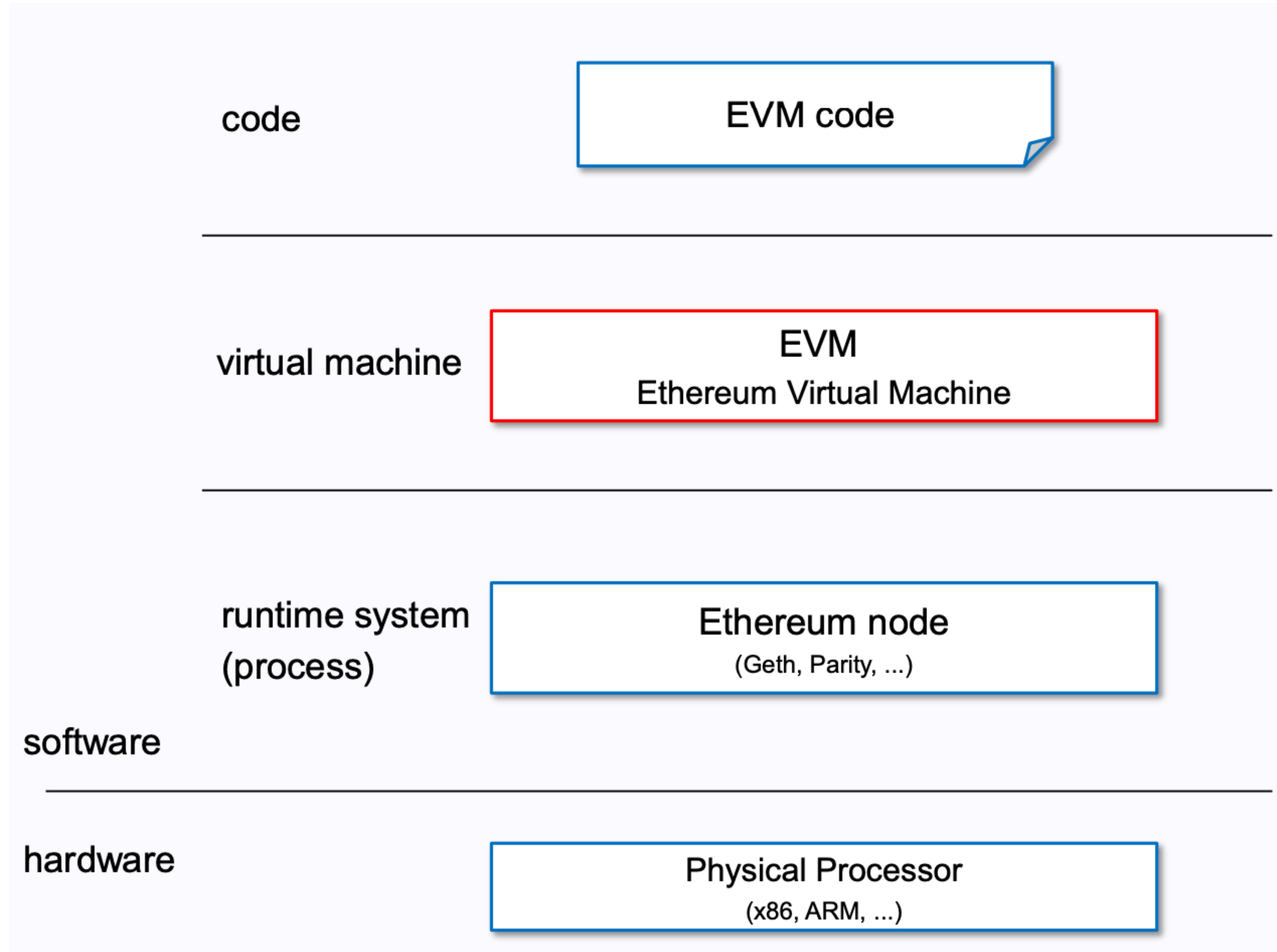
EVM



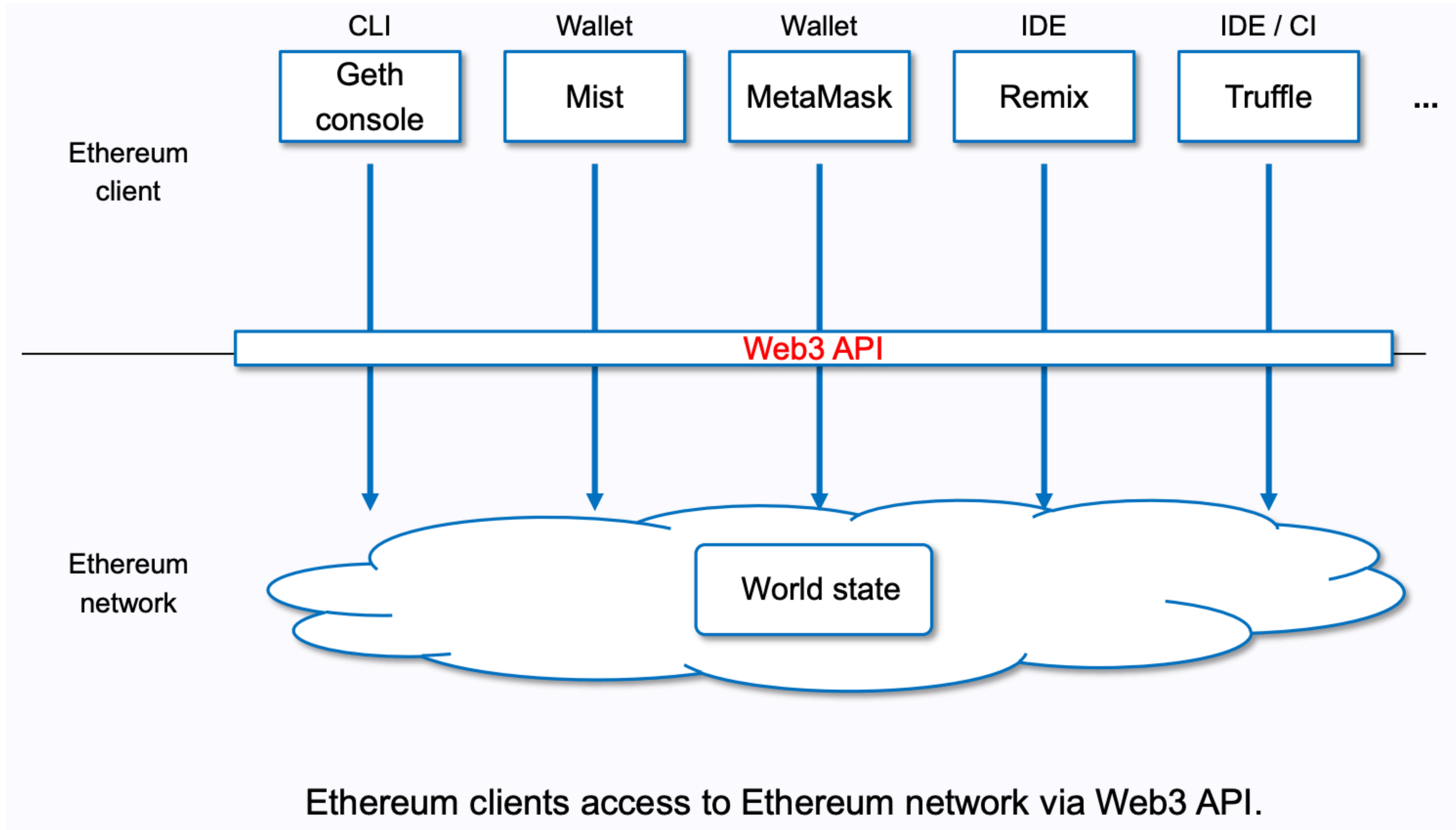
EVM



EVM



EVM



EVM

- Arithmetic Operations
- Stack Operations
- Process Flow Operations
- System Operations
- Logic Operations
- Environmental Operations
- Block Operations

EVM

Arithmetic Operations

ADD	//Add the top two stack items
MUL	//Multiply the top two stack items
SUB	//Subtract the top two stack items
DIV	//Integer division
SDIV	//Signed integer division
MOD	//Modulo (remainder) operation
SMOD	//Signed modulo operation
ADDMOD	//Addition modulo any number
MULMOD	//Multiplication modulo any number
EXP	//Exponential operation
SIGNEXTEND	//Extend the length of a two's complement signed integer
SHA3	//Compute the Keccak-256 hash of a block of memory

EVM

Stack Operations

POP	//Remove the top item from the stack
MLOAD	//Load a word from memory
MSTORE	//Save a word to memory
MSTORE8	//Save a byte to memory
SLOAD	//Load a word from storage
SSTORE	//Save a word to storage
MSIZE	//Get the size of the active memory in bytes
PUSHx	//Place x byte item on the stack, where x can be any integer from // 1 to 32 (full word) inclusive
DUPx	//Duplicate the x-th stack item, where x can be any integer from // 1 to 16 inclusive
SWAPx	//Exchange 1st and (x+1)-th stack items, where x can be any // integer from 1 to 16 inclusive

EVM

Process Flow Operations

STOP	//Halt execution
JUMP	//Set the program counter to any value
JUMPI	//Conditionally alter the program counter
PC	//Get the value of the program counter (prior to the increment

EVM

System Operations

LOGx	//Append a log record with x topics, where x is any integer //from 0 to 4 inclusive
CREATE	//Create a new account with associated code
CALL	//Message-call into another account, i.e. run another //account's code
CALLCODE	//Message-call into this account with another //account's code
RETURN	//Halt execution and return output data
DELEGATECALL	//Message-call into this account with an alternative //account's code, but persisting the current values for //sender and value
STATICCALL	//Static message-call into an account
REVERT	//Halt execution, reverting state changes but returning //data and remaining gas
INVALID	//The designated invalid instruction
SELFDESTRUCT	//Halt execution and register account for deletion

EVM

Logic Operations

LT	//Less-than comparison
GT	//Greater-than comparison
SLT	//Signed less-than comparison
SGT	//Signed greater-than comparison
EQ	//Equality comparison
ISZERO	//Simple NOT operator
AND	//Bitwise AND operation
OR	//Bitwise OR operation
XOR	//Bitwise XOR operation
NOT	//Bitwise NOT operation
BYTE	//Retrieve a single byte from a full-width 256-bit word

EVM

Environmental Operations

GAS	//Get the amount of available gas (after the reduction for //this instruction)
ADDRESS	//Get the address of the currently executing account
BALANCE	//Get the account balance of any given account
ORIGIN	//Get the address of the EOA that initiated this EVM //execution
CALLER	//Get the address of the caller immediately responsible //for this execution
CALLVALUE	//Get the ether amount deposited by the caller responsible //for this execution

EVM

Block Operations

BLOCKHASH	//Get the hash of one of the 256 most recently completed //blocks
COINBASE	//Get the block's beneficiary address for the block reward
TIMESTAMP	//Get the block's timestamp
NUMBER	//Get the block's number
DIFFICULTY	//Get the block's difficulty
GASLIMIT	//Get the block's gas limit

EVM - Gas

- Adding two numbers costs 3 gas
- Calculating a Keccak-256 hash costs 30 gas + 6 gas for each 256 bits of data being hashed
- Sending a transaction costs 21,000 gas

EVM - Gas

If the EVM reaches the end of execution successfully, without running out of gas, the gas cost used is paid to the miner as a transaction fee, converted to ether based on the gas price specified in the transaction:

$$\text{miner fee} = \text{gas cost} * \text{gas price}$$

The gas remaining in the gas supply is refunded to the sender, again converted to ether based on the gas price specified in the transaction:

$$\begin{aligned}\text{remaining gas} &= \text{gas limit} - \text{gas cost} \\ \text{refunded ether} &= \text{remaining gas} * \text{gas price}\end{aligned}$$

EVM - Gas

While the gas *cost* is a measure of computation and storage used in the EVM, the gas itself also has a *price* measured in ether. When performing a transaction, the sender specifies the gas price they are willing to pay (in ether) for each unit of gas, allowing the market to decide the relationship between the price of ether and the cost of computing operations (as measured in gas):

$$\text{transaction fee} = \text{total gas used} * \text{gas price paid (in ether)}$$

EVM - Gas

For example, let's say we have 5 transactions whose gas limits have been set to 30,000, 30,000, 40,000, 50,000, and 50,000. If the block gas limit is 180,000, then any four of those transactions can fit in a block, while the fifth will have to wait for a future block. As previously discussed, miners decide which transactions to include in a block. Different miners are likely to select different combinations, mainly because they receive transactions from the network in a different order.

Blockchain Scalability

Blockchain-based systems currently face significant limitations in transaction capacity when compared to traditional payment networks.

Comparison of Average Transaction Rates¹

- **Bitcoin** 4 Tx/sec
- **Ethereum** 20 Tx/sec
- **Paypal** 193 Tx/sec
- **Visa** 1,667 Tx/sec

Traditional Payment Networks handle a high volume of transactions through centralized infrastructure, which optimizes network efficiency. This results in low and stable transaction fees.

Blockchain-based Systems are limited by decentralized network constraints, leading to lower transaction capacity. As user demand exceeds network capacity, transaction fees increase significantly, causing higher fees and longer confirmation times.

Blockchain Scalability

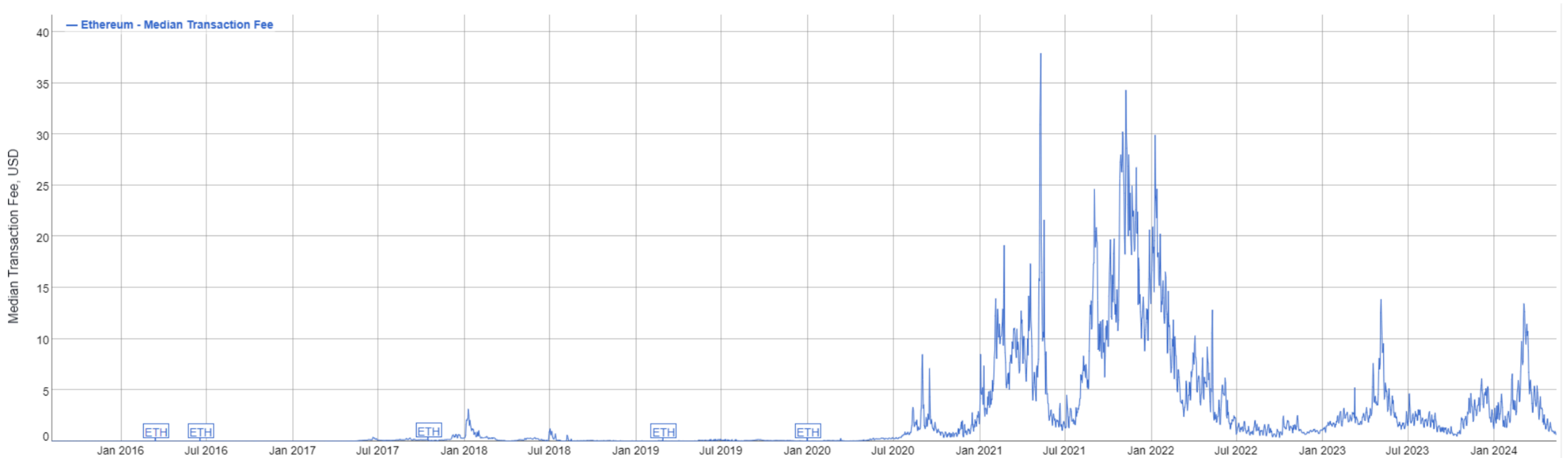


Image Source - <https://bitinfocharts.com/comparison/ethereum-transactionfees.html>

Factors Influencing Transaction Fees

- **Block Capacity:** Limited transaction capacity per block creates a competitive fee market.
- **Rising Network Activity:** DeFi and NFTs drive increased transactions, leading to higher fees.

Blockchain Scalability

Scalability Solutions aim to reduce high transaction fees and long confirmation times by increasing network capacity.

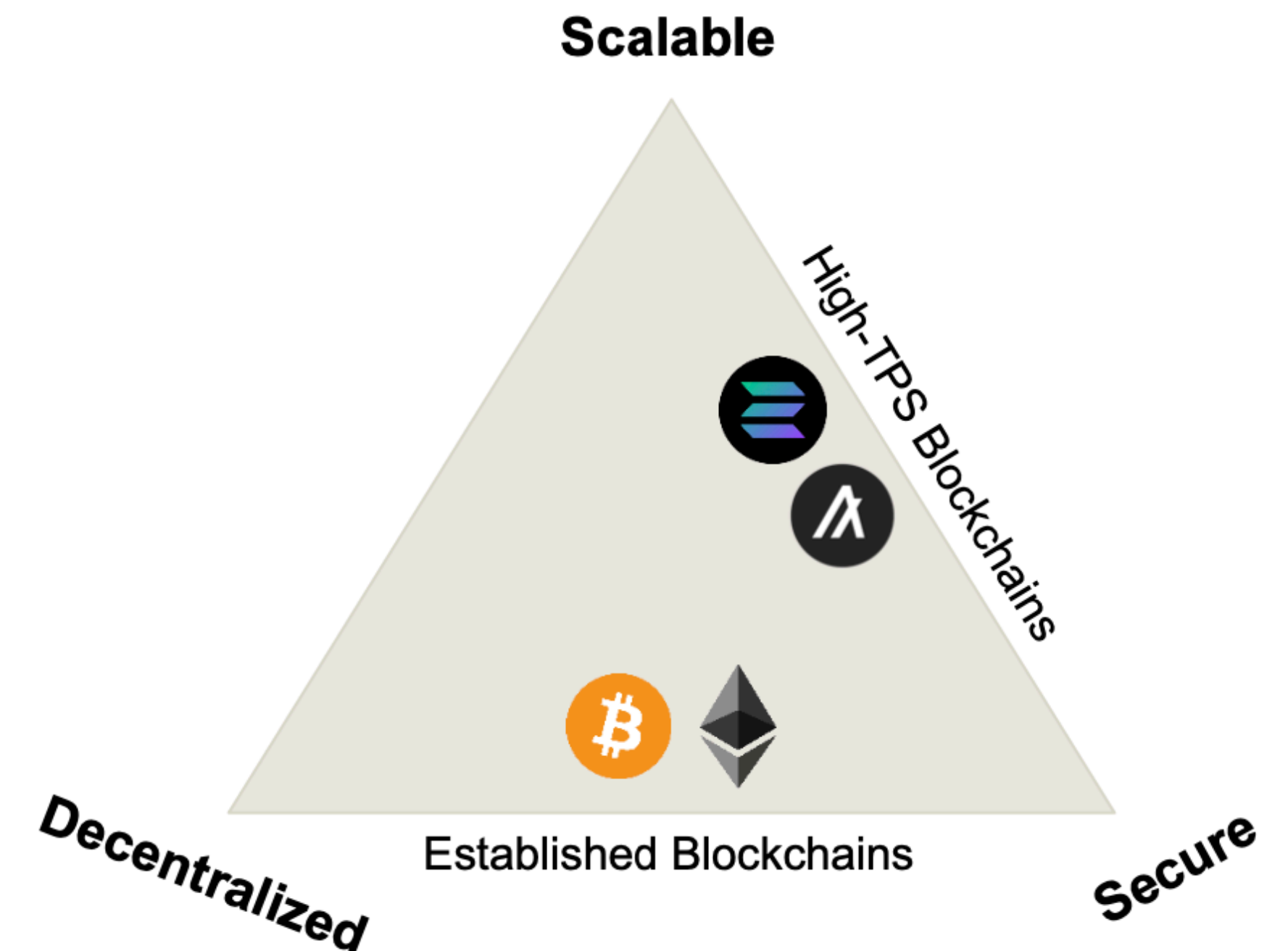
The Scalability Trilemma highlights the difficulty of enhancing blockchain performance without compromising other aspects. Achieving decentralization, security, and scalability simultaneously is challenging, with improvements in one often requiring trade-offs among the others.

Features of the Trilemma

- **Scalability:** Ability to handle increasing transactions.
- **Security:** Ensuring integrity and resistance to attacks.
- **Decentralization:** Distributing power to avoid central control.

Balancing Act

Maintaining a balance between scalability, security, and decentralization is crucial for blockchain adoption. Developers continuously seek ways to improve scalability without compromising the other two aspects.



Blockchain Scalability

To address increasing transaction fees in blockchain networks, vertical scaling offers a straightforward solution by tweaking core blockchain parameters.

Method: Increase block size and reduce block time.

Result: Validators process more transactions faster.

Advantages of Vertical Scaling

- Lowers transaction fees and boosts network capacity.
- Supports real-time applications like high-frequency trading and gaming.
- Makes blockchain viable for microtransactions and daily use.

Examples

Algorand and Solana can sustain high transaction speeds using advanced hardware.

- **Solana** 2390 Tx/sec¹
- **Algorand** 2300 Tx/sec²



Blockchain Scalability

While vertical scaling can enhance blockchain performance, there are technical limitations.



Technical Limitations

- **Compute:** Consumer hardware has limited processing power.
- **Bandwidth:** Larger blocks could slow down nodes in regions with lower internet speeds.
- **Storage:** Bigger blocks need more disk space, making it harder for average users to run nodes.

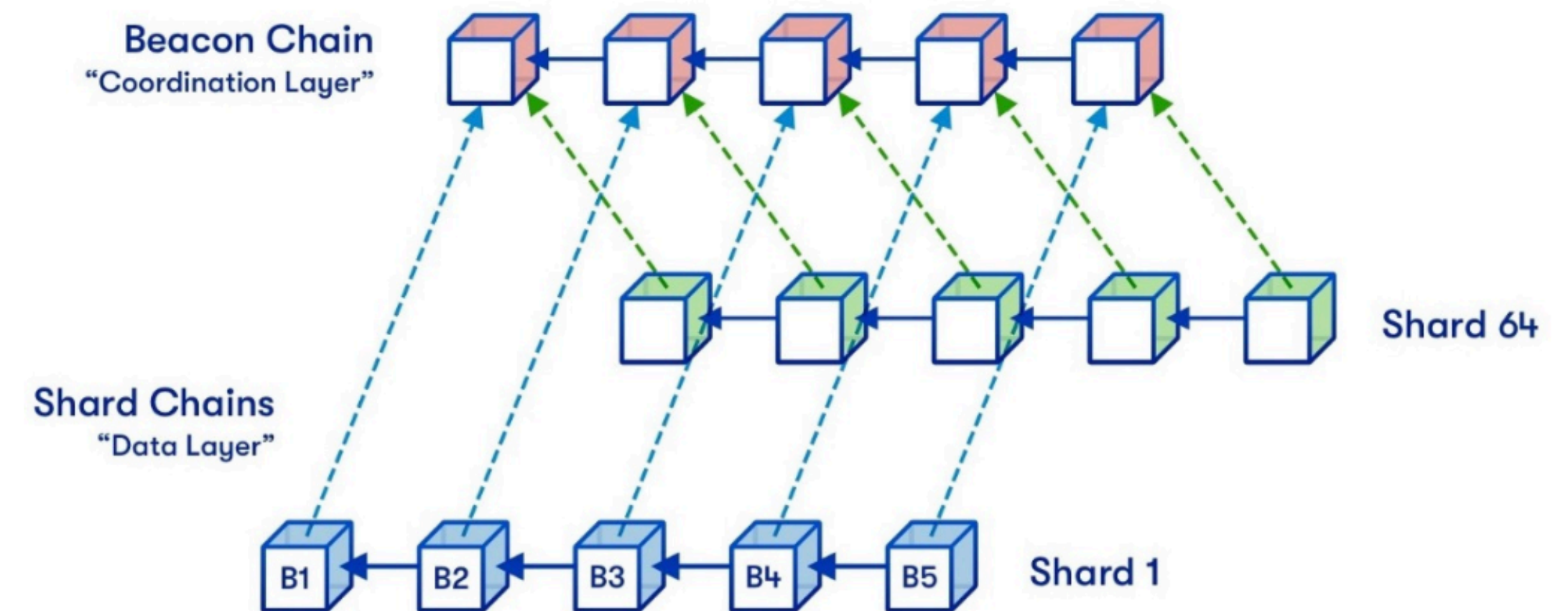
Risk to Decentralization

Vertical scaling could centralize the network by excluding users who lack high-performance hardware. This reduces overall participation and creates a barrier to entry for joining the network.

Blockchain Scalability

Sharding is a scaling solution that increases transaction throughput without increasing demands on individual nodes.

- **Network Division:** The blockchain network is divided into smaller segments called shards.
- **Parallel Processing:** Each shard processes its own set of transactions, allowing multiple transactions to be handled simultaneously.
- **Coordination:** Shards communicate and coordinate to maintain network consistency and security.



Sharding in the Context of the Blockchain Scalability Trilemma

- **Scalability:** Shards work independently, handling many more transactions than a single node could.
- **Decentralization:** Shards can run on regular consumer hardware, avoiding the need for high-performance hardware.
- **Security:** Spreading validation across many shards makes it harder for attackers to compromise the network.

Blockchain Scalability

Blockchain Scalability Trilemma Solved?
Not Quite.

While sharding offers significant benefits, it also introduces technical complexities for developers and the blockchain architecture that the blockchain trilemma doesn't account for.

Challenges of Implementing Sharding

- **Data Consistency:** Ensuring data remains consistent across different shards.
- **Cross-Shard Communication:** Efficiently coordinating transactions between different shards.
- **Data Availability:** All necessary data is accessible and verifiable without requiring every node to download the entire data.

Polkadot is one example of a blockchain that has successfully implemented sharding to enhance scalability.

For more detailed information on sharding, consider exploring these resources:

- <https://vitalik.eth.limo/general/2021/04/07/sharding.html>
- https://hackmd.io/@vbuterin/sharding_proposal



Blockchain Scalability

The idea of Layer 2 Scaling is to build on foundations of existing blockchains.

Scaling the blockchain by handling transactions off the main blockchain (Layer 1), reducing load and increasing efficiency without sacrificing security.

Payment Channels: Private pathways between two parties for off-chain transactions. The main blockchain only records the channel's opening and closing, lowering the number of on-chain transactions.

- E.g., Bitcoin Lightning Network (multiple different implementations)



Scaling via Rollups: Rollups process transactions off-chain but commit data back to the main chain for verification.

- Rollup Networks on Ethereum: Optimism¹, Arbitrum², zkSync³, Base⁴

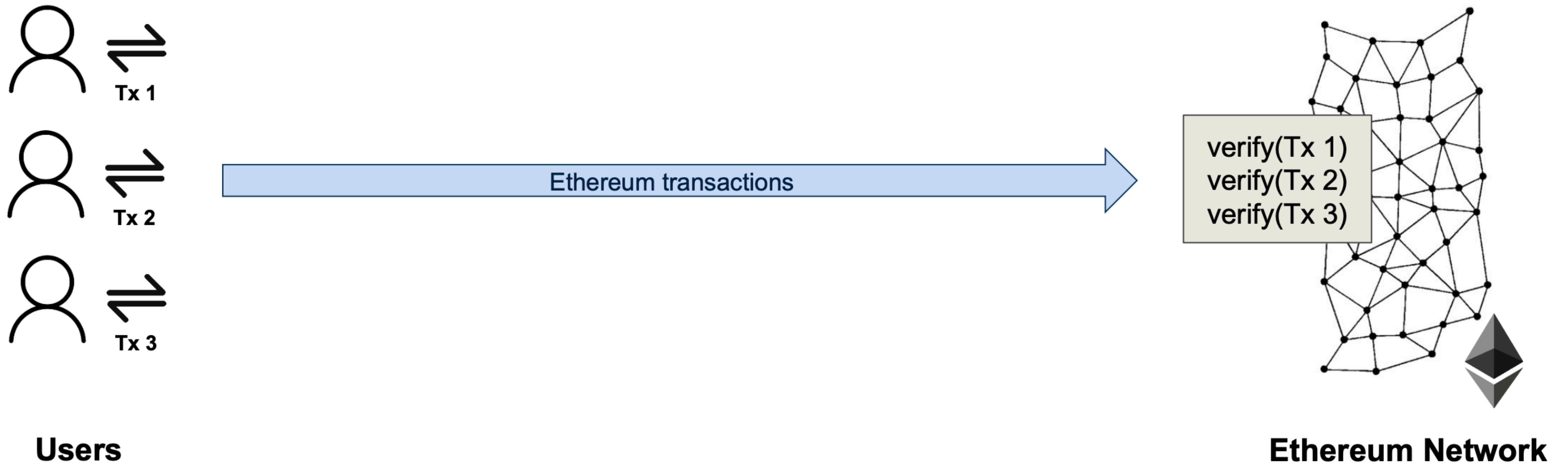


zkSync



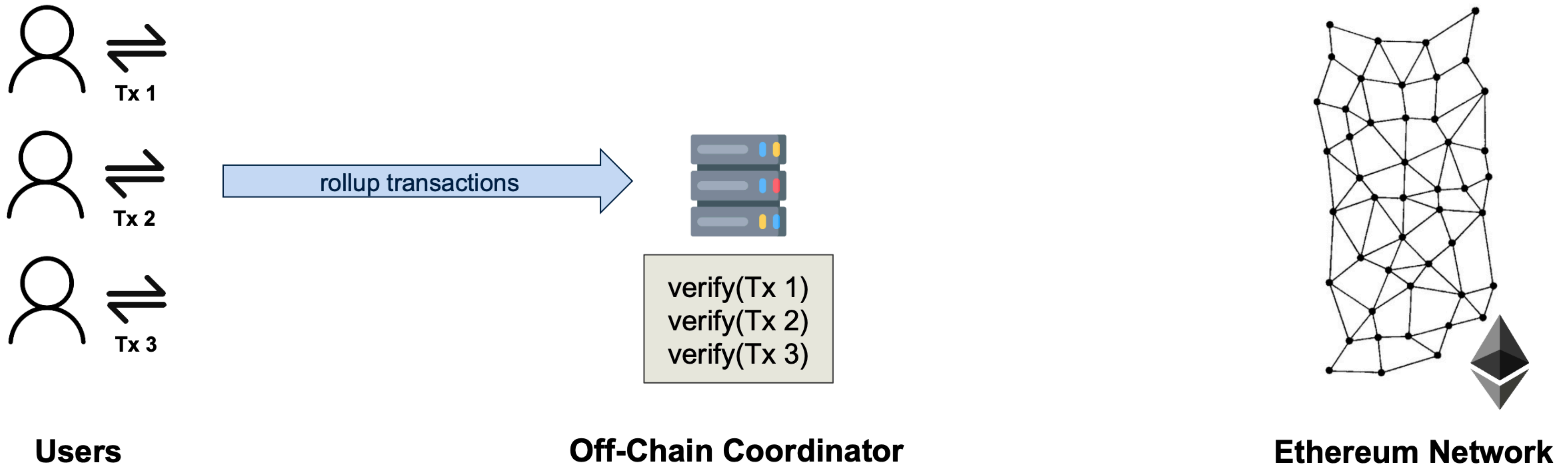
Blockchain Scalability

- Each transaction on Ethereum must be validated by all validators.
- This slows down the network, limiting transaction throughput.



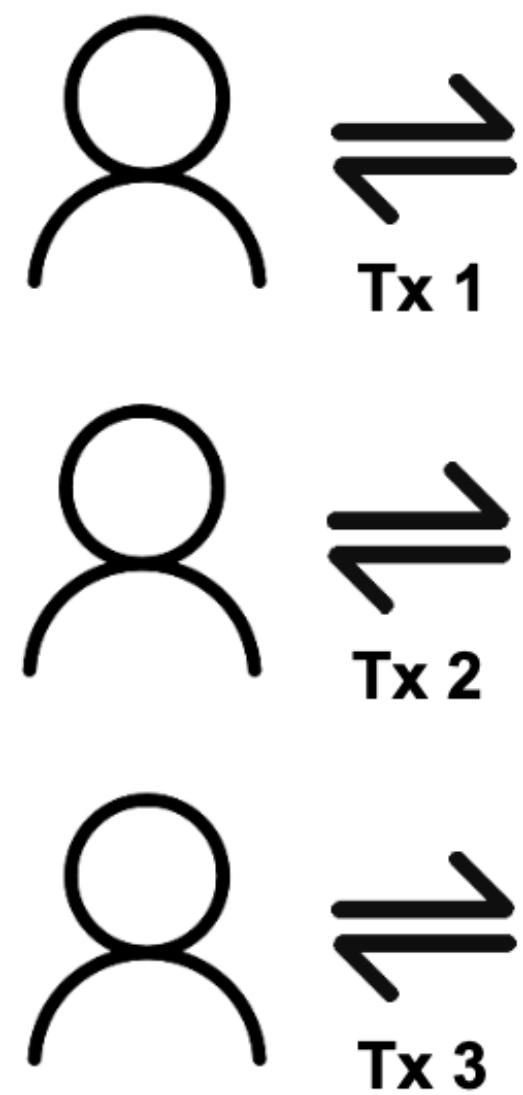
Blockchain Scalability

- Rollups introduce a new party, a single **off-chain coordinator**.
- Users send **rollup-transactions** to the coordinator instead of Ethereum transactions.
- The coordinator **checks the transactions for validity and attests** that all transactions are valid.

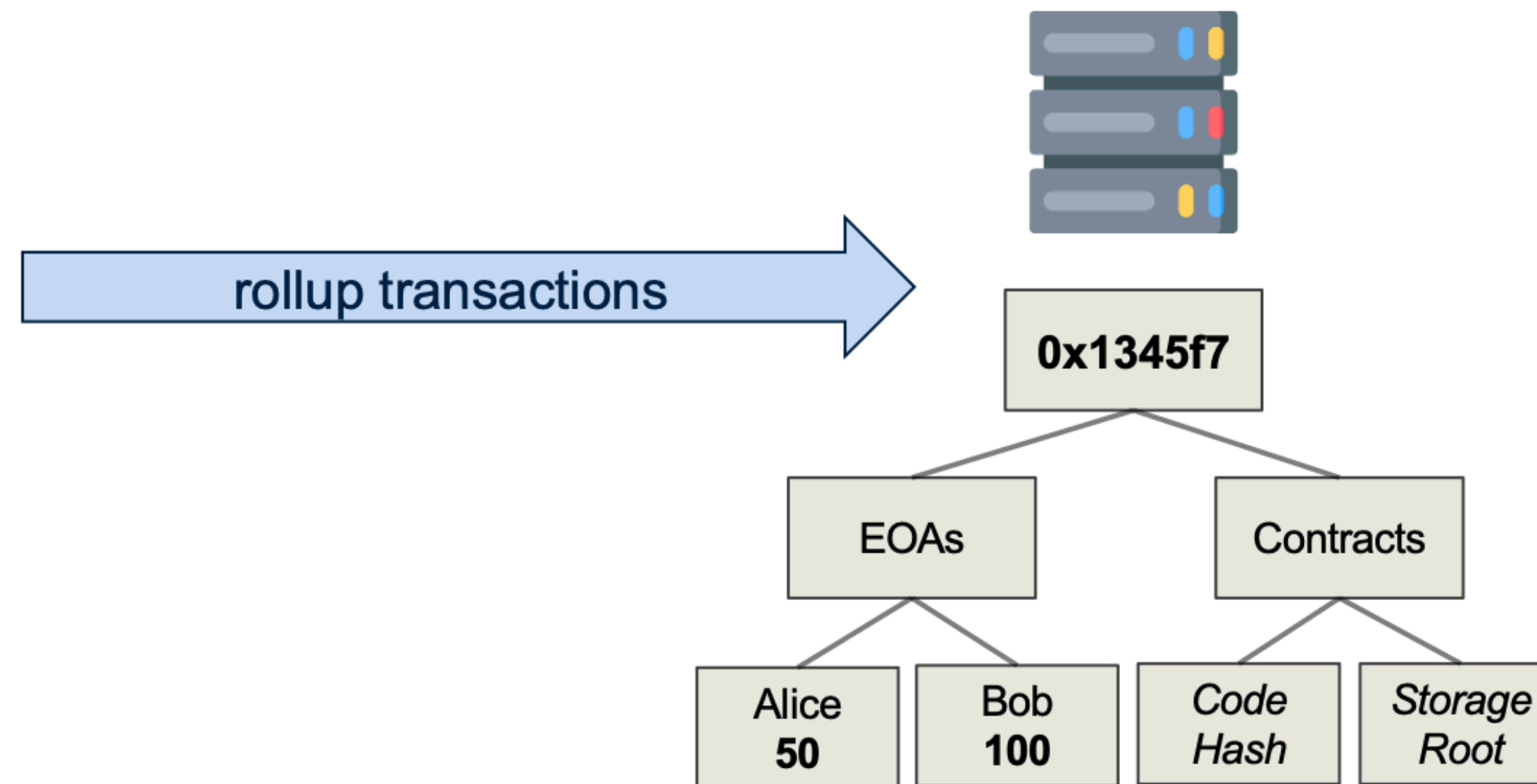


Blockchain Scalability

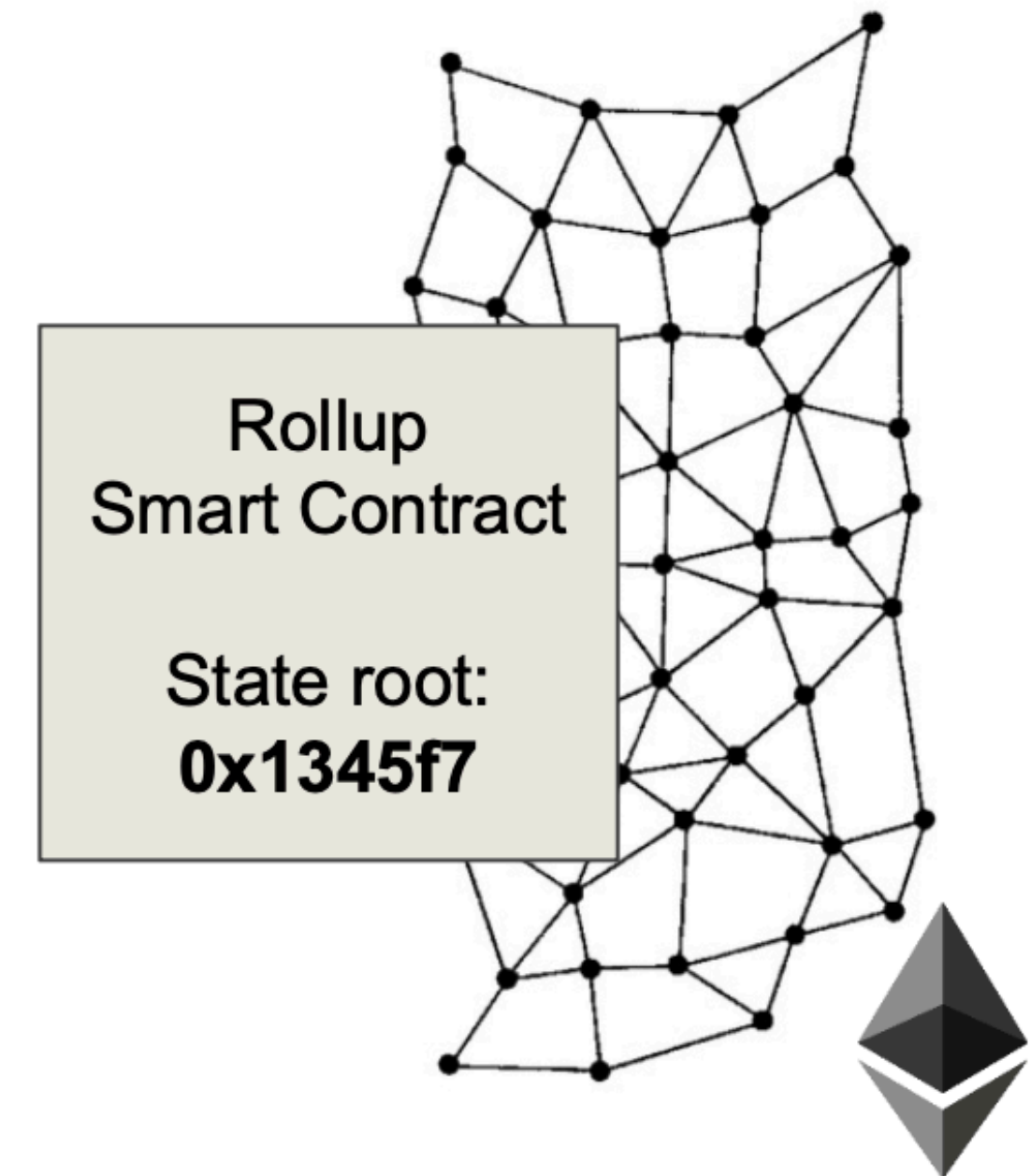
- The coordinator manages an independent blockchain and creates a Merkle tree of its state.
- The coordinator writes the tree's root into a smart contract on Ethereum.
 - Ensures the integrity of all off-chain transactions



Users



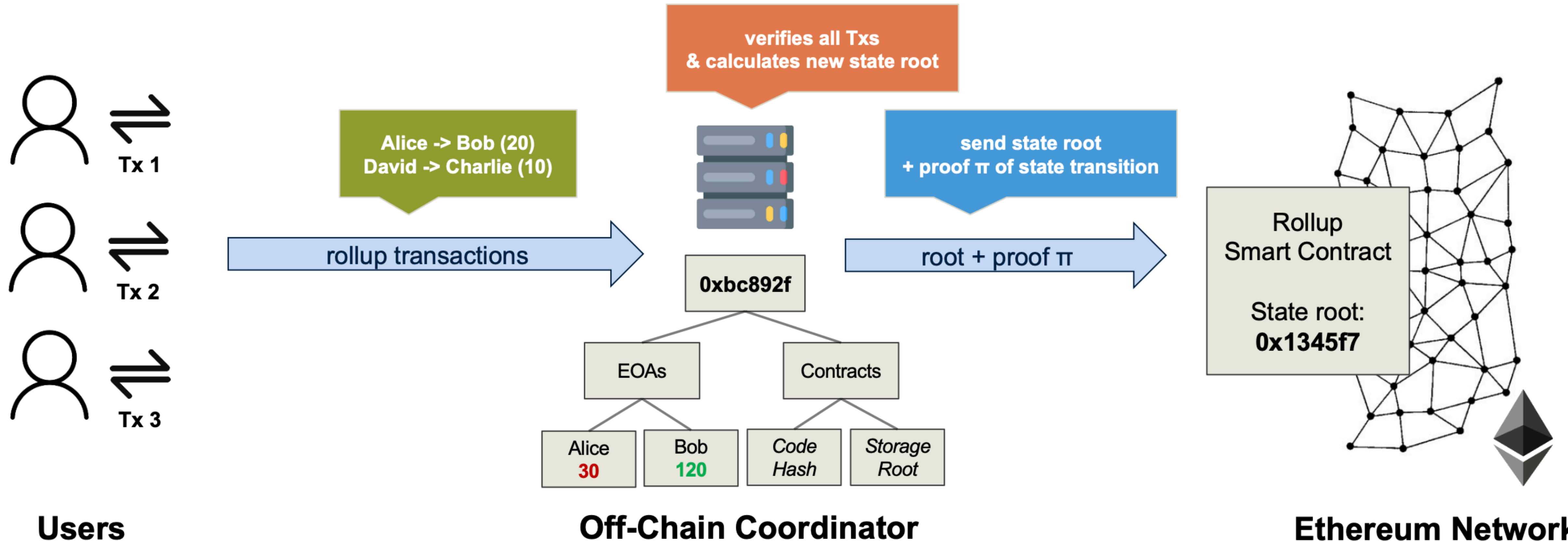
Off-Chain Coordinator



Ethereum Network

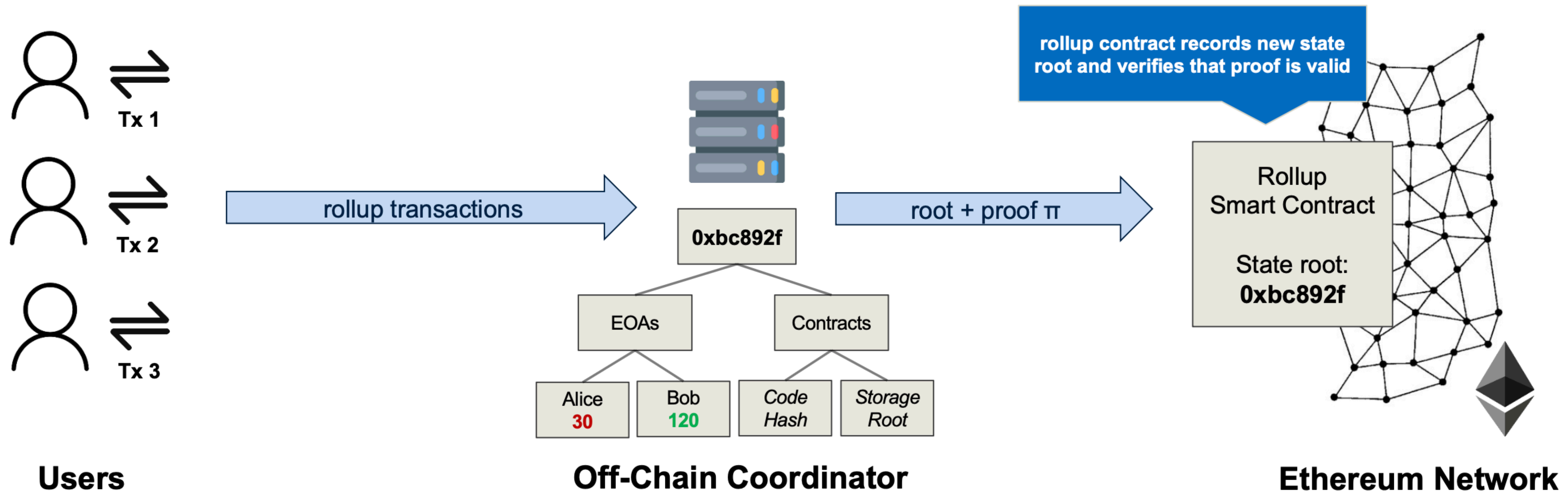
Blockchain Scalability

1. Users **send (rollup)-transactions** to the coordinator.
2. The coordinator **updates account balances** and **calculates a new Merkle tree root**.
3. The coordinator generates a proof of the validity of the new Merkle-tree root.
4. The coordinator **sends the new root and proof to Ethereum contract**.



Blockchain Scalability

- The rollup coordinator updates the new state root in the rollup contract along with a proof of integrity.
 - The rollup contract records the new state root and verifies that the proof is valid.
- ✓ The amount of data posted to the blockchain is minimal (spreading costs across multiple transactions).



Blockchain Scalability

How can the rollup contract check if the state root transition proof is valid?

There are two main approaches: **Optimistic Rollups** and **Zero-Knowledge Rollups**.

Optimistic Rollups (e.g., Optimism Implementation¹)

- Transaction data sent every ~10 minutes to the rollup contract.
- Rollup assumes all transactions to be valid unless proven otherwise.
- Auditors have a challenge period of 1 week to dispute a batch by providing a fraud proof on chain.
- After the challenge period, the state root is updated on the contract.



Zero-Knowledge Rollups (e.g., zkSync Era Implementation²)

- Proof consists of a cryptographic proof of the validity for each transaction batch.
- Proofs are stored and immediately verified on the blockchain, eliminating the need for a challenge period.
- zkSync Era mainnet already supports zero knowledge proofs for EVM compatible rollups.
- Although at this point only with a centralized coordinator.



Blockchain Scalability

Property	Optimistic Rollup (Optimism)	Zero Knowledge Rollup (zkSync Era)
Fixed Gas Usage per Transaction Batch	~40,000 ² (lightweight transaction)	~500,000 ² (computationally intensive verification)
Withdrawal Period	~1-week ¹ (allows time for fraud proofs)	Immediate (wait for the next batch)
Contract Complexity	Low	High (complex ZK cryptography)
Off-chain computation costs	Lower	Very High (ZK proving can be very expensive)

Blockchain Interoperability

Isolation of Funds

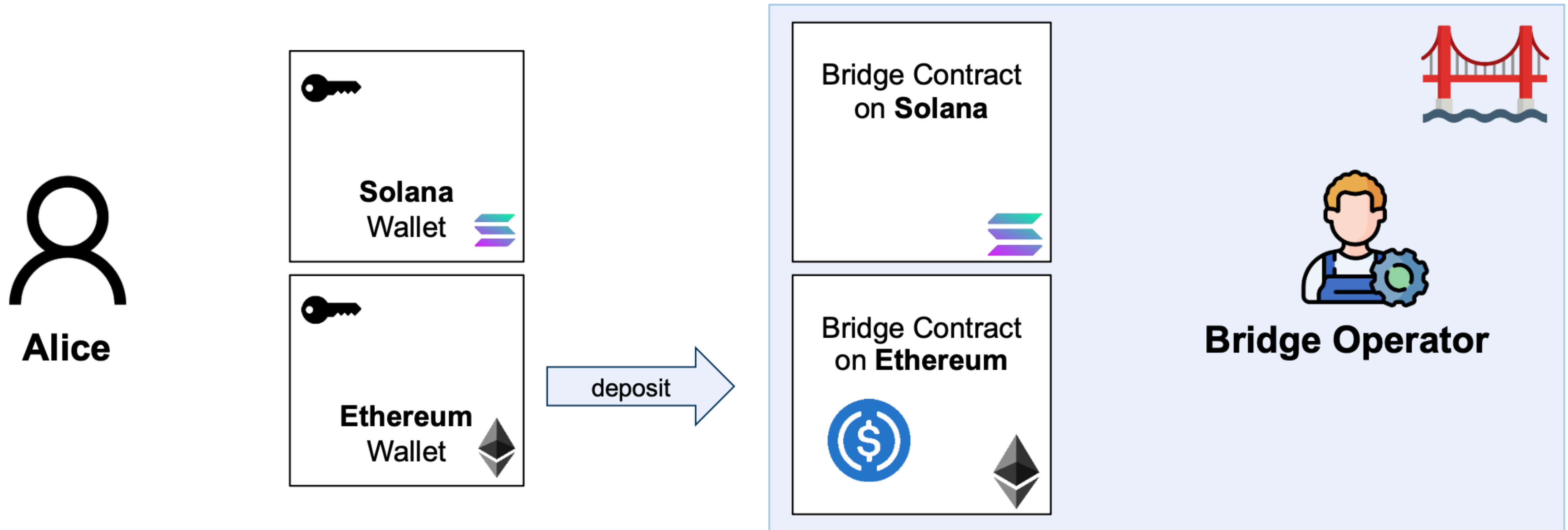
- Funds on one chain or rollup cannot be easily transferred to another, leading to isolated and less efficient blockchain projects.
- This isolation hinders the growth and full potential of decentralized networks.

Blockchain Interoperability

- The ability of different blockchain networks to communicate, share data, and build on each other's features.
- Enables the seamless exchange of data and assets without intermediaries.

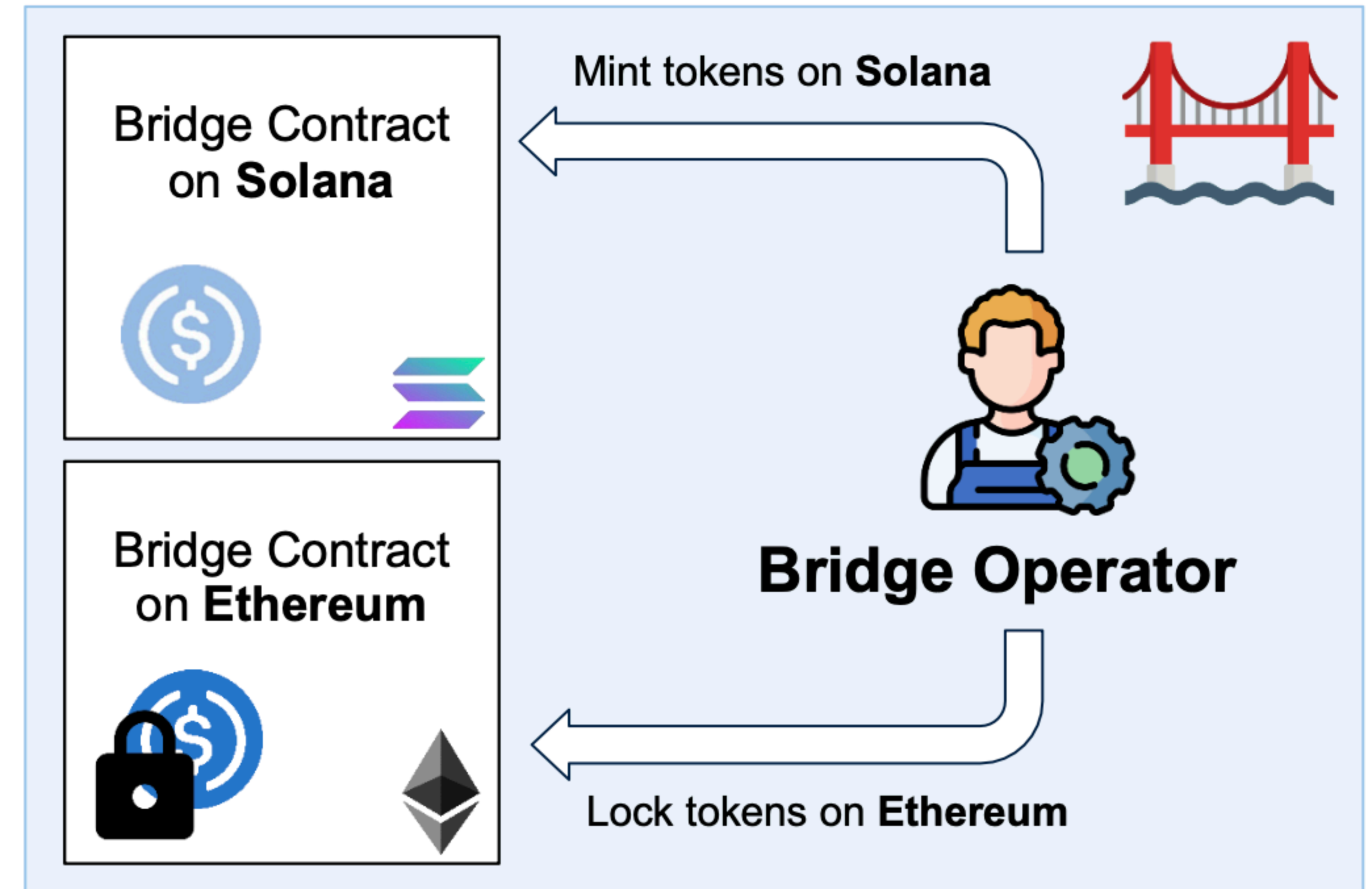
Blockchain Interoperability

- Facilitate the transfer of value between different blockchain networks.
- Enable interoperability by locking tokens on the source chain and minting equivalent tokens on the destination chain.



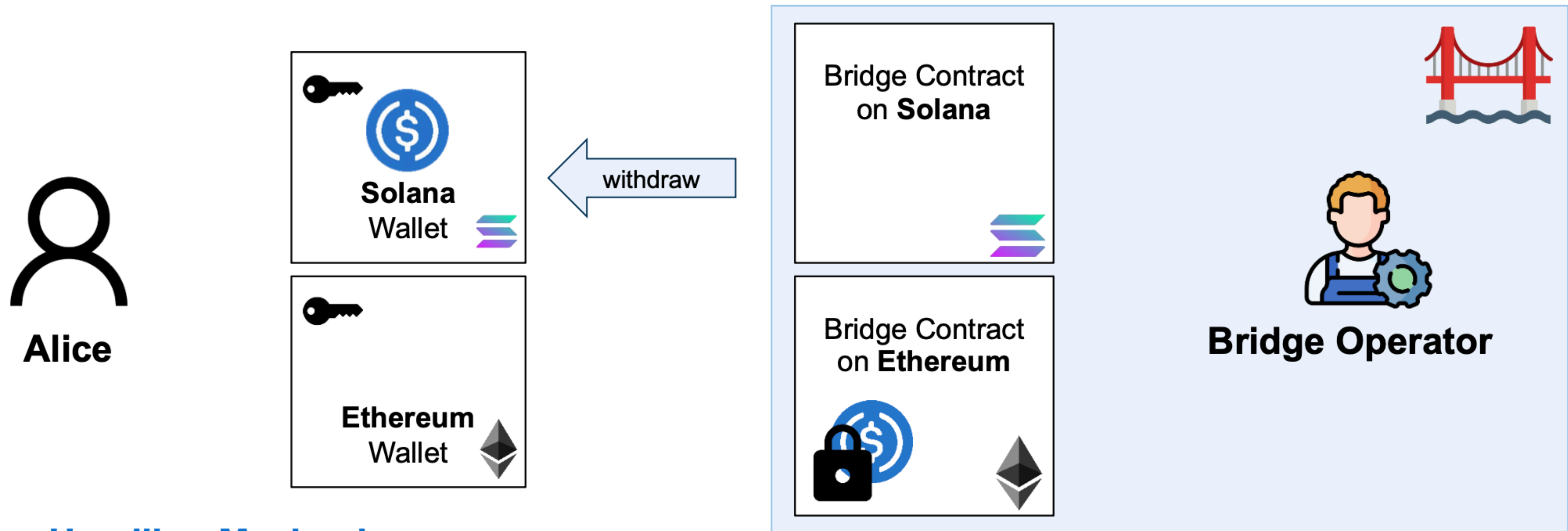
Blockchain Interoperability

- Facilitate the transfer of value between different blockchain networks.
- Enable interoperability by locking tokens on the source chain and minting equivalent tokens on the destination chain.



Blockchain Interoperability

- Facilitate the transfer of value between different blockchain networks.
- Enable interoperability by locking tokens on the source chain and minting equivalent tokens on the destination chain.

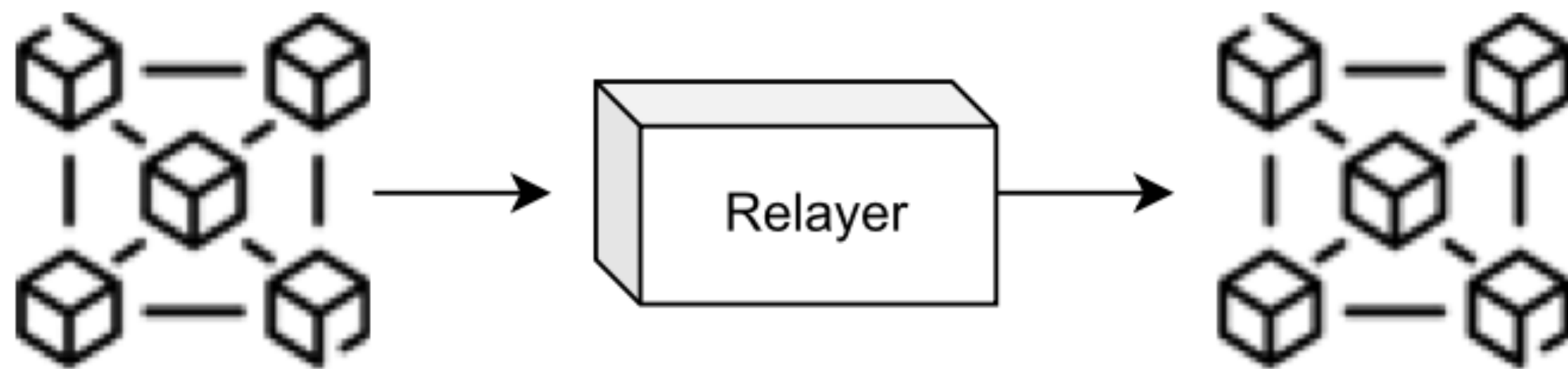


Token Handling Mechanisms

- **Lock and Mint:** Lock tokens on the source chain and mint equivalent tokens on the destination chain.
- **Burn and Mint:** Burn tokens on the source chain and mint equivalent tokens on the destination chain.
- **Lock and Release:** Lock tokens on the source chain and release tokens from a reserve on the destination chain.

Blockchain Interoperability Patterns

Relayer Pattern*



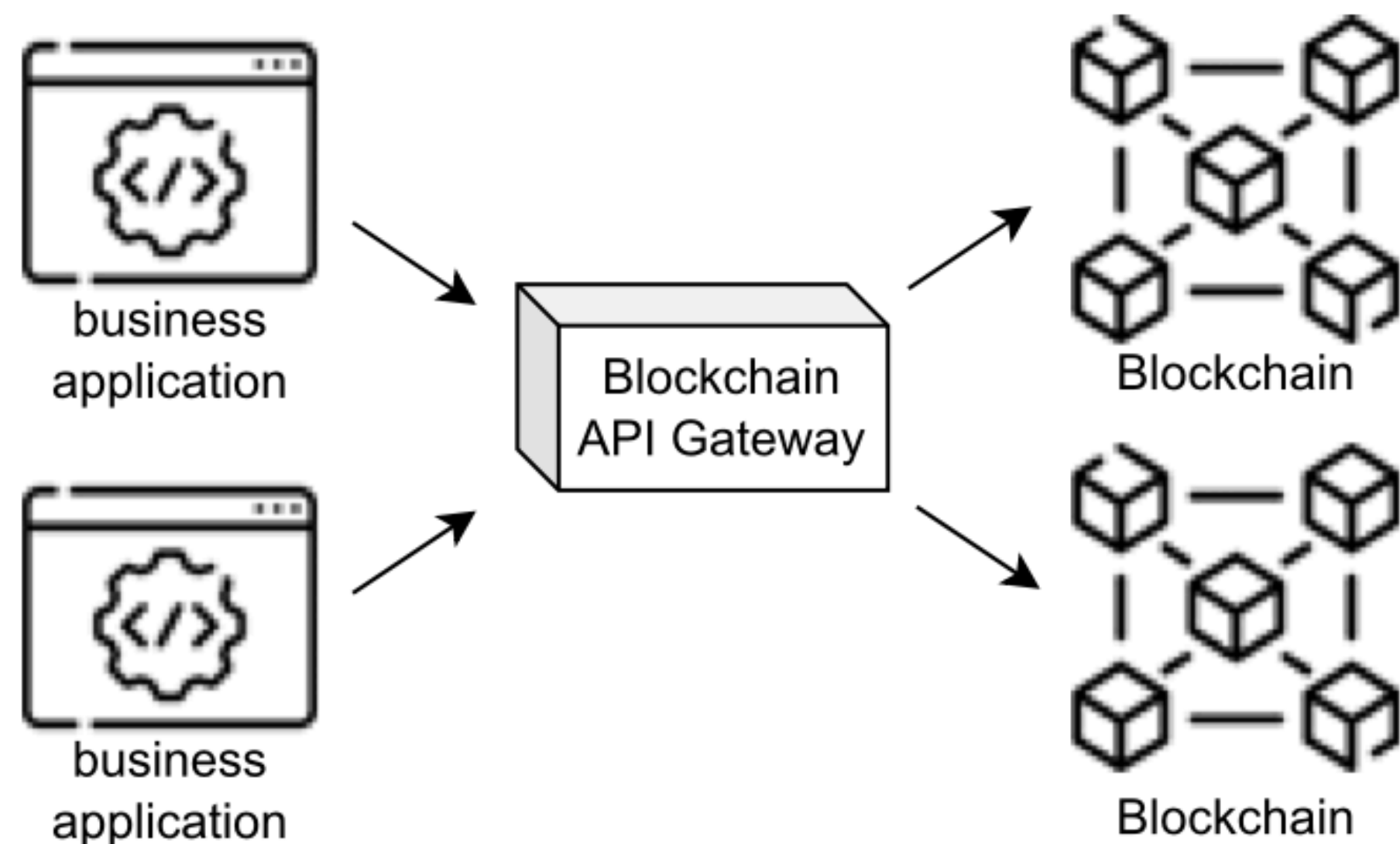
Summary: A third-party middleware (centralised or decentralised) called Relayer enables to send the state of an executed transaction from a source blockchain to a target blockchain. Blockchains may be homogeneous or heterogeneous. Fig. 3 presents a graphical representation of the pattern.

Context: A target blockchain needs the state of an executed transaction on a source blockchain to perform a task (e.g. execute a smart contract).

* [Paper] Blockchain interoperability patterns,...

Blockchain Interoperability Patterns

API Gateway Pattern

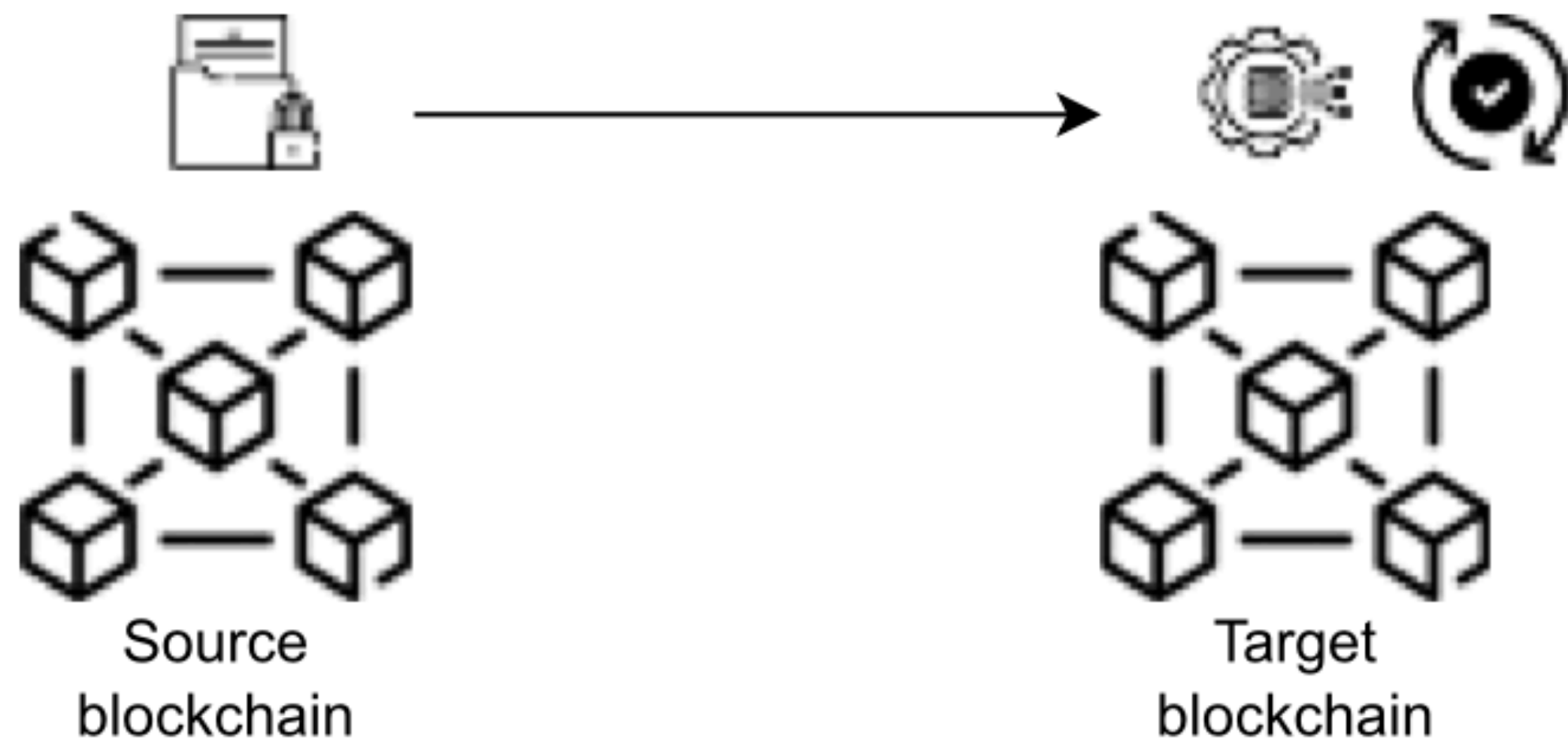


Summary: A third-party middleware called API Gateway simplifies the connectivity of one or more traditional business applications with one or more blockchains through an API. Fig. 4 presents a graphical representation of the pattern.

Context: A business application needs to communicate with one or more blockchains to complete its business process. For example, to submit transactions or query the blockchain.

Blockchain Interoperability Patterns

Light Client Pattern

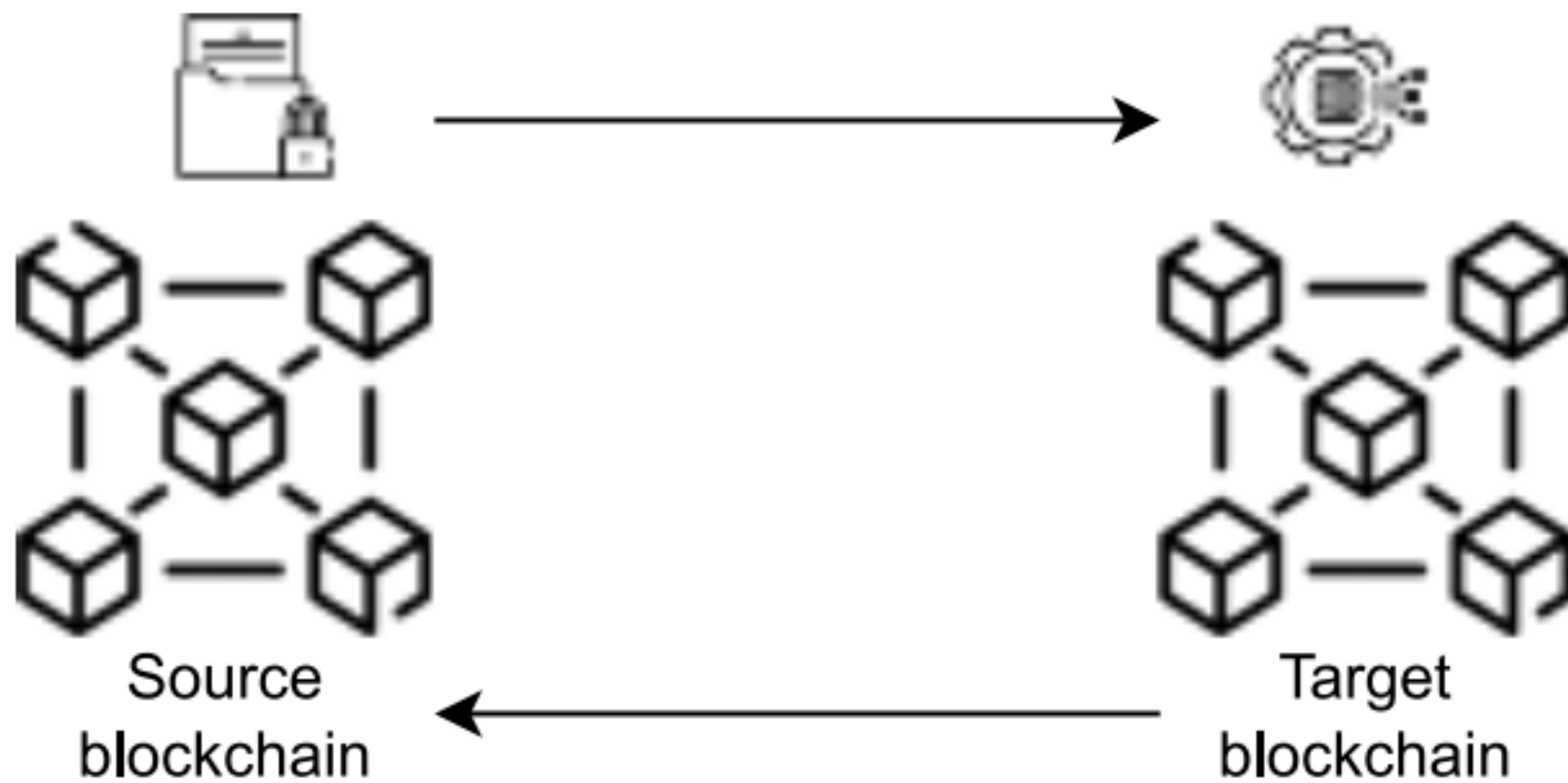


Summary: A smart contract on the target blockchain stores a trusted consensus state of a source blockchain and provides operations to verify data against this state. Fig. 5 presents a graphical representation of the pattern.

Context: A source blockchain sends data to a target blockchain using an interoperability mechanism (e.g. applying the Relayer pattern). Blockchains may use different consensus protocols, but support the same technical capabilities (e.g. same hash functions and signature algorithms).

Blockchain Interoperability Patterns

Temporal Asset Transfer Pattern

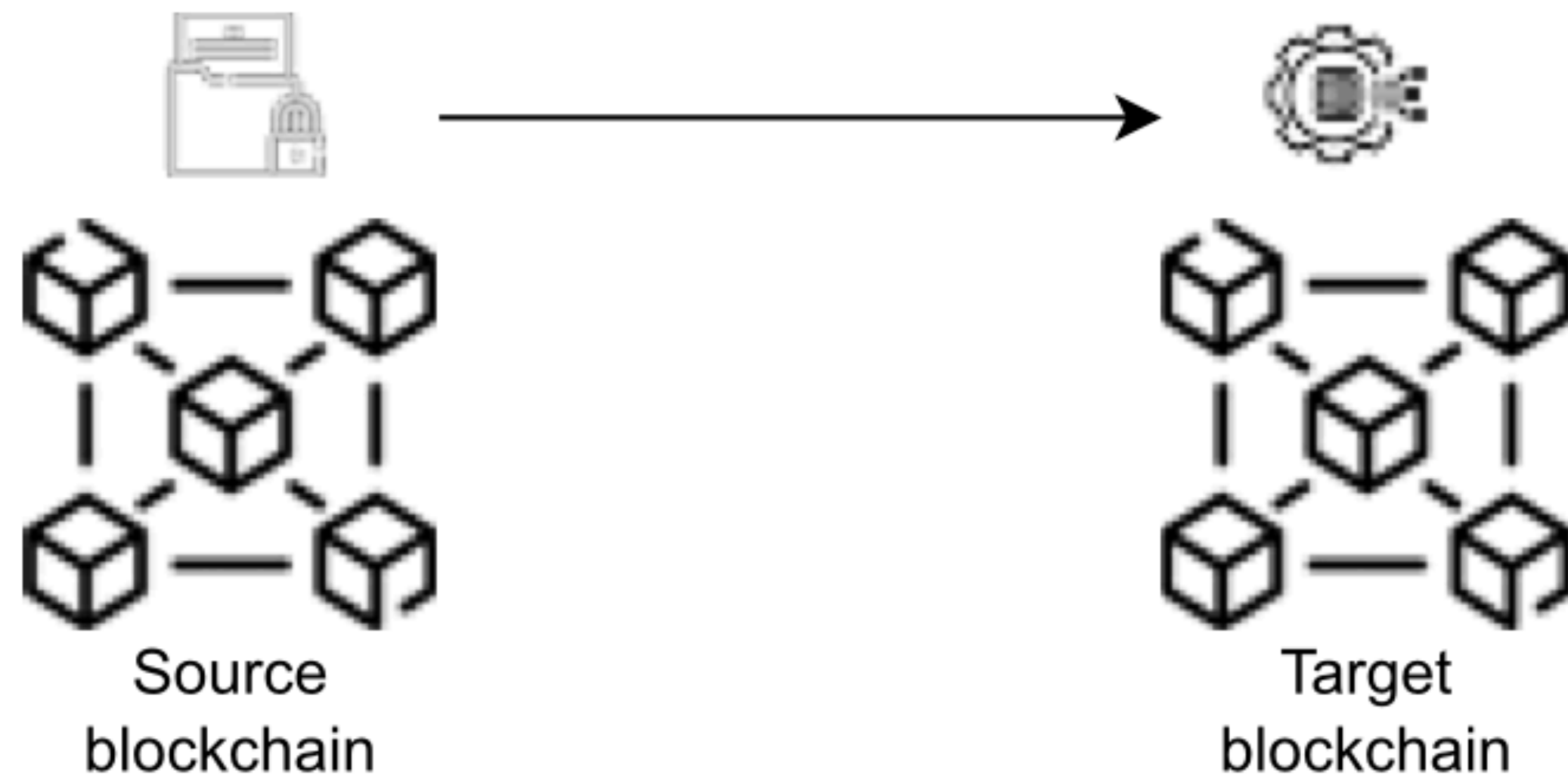


Summary: An asset hosted on a source blockchain is temporary transferred to a target blockchain. That asset can be temporary used in the target blockchain until it is returned back to the source blockchain. Fig. 6 presents a graphical representation of the pattern.

Context: Blockchains execute transactions with different costs and performance. In addition, blockchains have different characteristics and not every blockchain supports the same features. Blockchains have technical compatibility (e.g. same hash functions and signature algorithms) that enables them to semantically understand shared data. A user may want to temporary move an asset from a source blockchain to a target blockchain to have some benefit of these blockchain heterogeneity (e.g. execute transactions at less costs).

Blockchain Interoperability Patterns

Permanent Asset Transfer Pattern

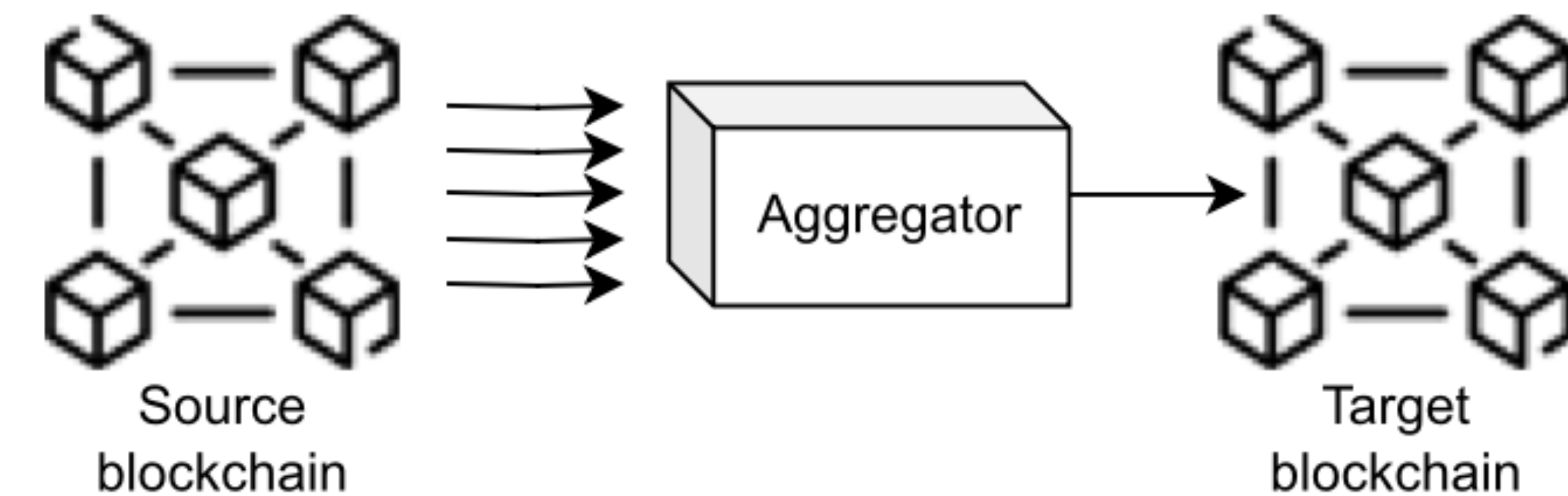


Summary: An asset hosted on a blockchain is permanently transferred to another blockchain. Fig 7 presents a graphical representation of the pattern.

Context: A user has an asset on a blockchain and does not want to use that blockchain anymore. This may happen in case another blockchain provides better features, less costs or higher performance, among other reasons. Blockchains have technical compatibility (e.g. same hash functions and signature algorithms) that enables them to semantically understand shared data.

Blockchain Interoperability Patterns

Aggregator Transfer Pattern



Summary: A third party middleware called Aggregator listens to the execution of transactions on a source blockchain and packages a large set of these transactions into a compressed dataset. This compressed dataset is submitted as a unique transaction to a target blockchain. Fig 7 presents a graphical representation of the pattern.

Context: The source blockchain needs to send the state of a large set of executed transactions to a target blockchain.

Blockchain Interoperability

Bridges can be major targets due to central points of control.
Bugs or exploits in bridge contracts can lead to significant losses.

Wormhole Bridge Hack

In February 2022, the Wormhole bridge was hacked for \$325 million due to a security flaw.

Method

- The attacker forged a valid signature.

Impact

- Minted 120,000 wETH on the Solana blockchain.
- Exchanged it for \$250 million in ETH.

TECH / SECURITY / POLICY

Wormhole cryptocurrency platform hacked for \$325 million after error on GitHub

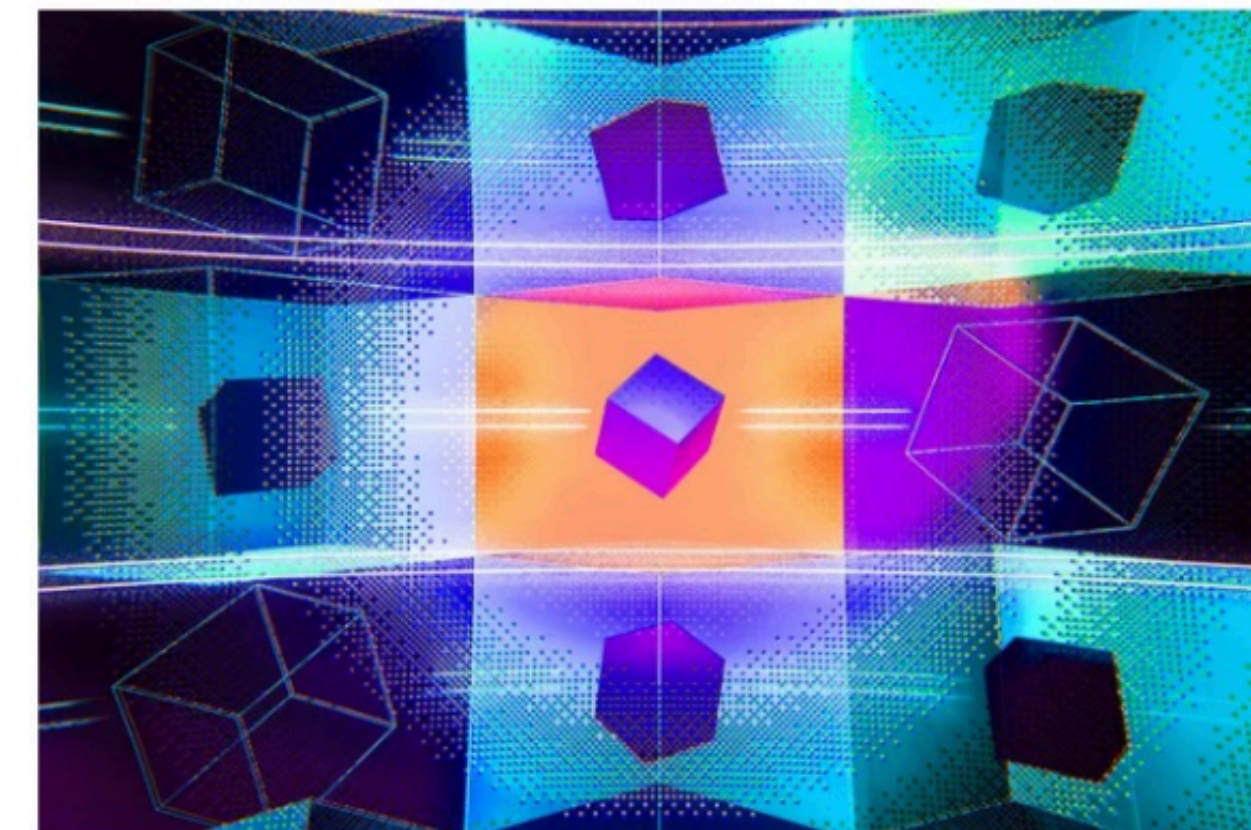


Illustration by Alex Castro / The Verge

/ A security flaw was fixed but seemingly not applied to the live application before it was hacked

By [Corin Faife](#)

Feb 3, 2022, 6:43 PM GMT+1



0 Comments (0 New)